

SUBMITTED BY:

Name:	Yaswanth Varun. A
Roll no:	AV.SC.U4CSE25004
Branch:	CSE
Section:	CSE-A

SUBMITTED TO:

Name:	DR. Raj Kumar Batchu
Department:	School Of Computing
Designation:	Asst. Proffesor

Subject: Data Structures and Algorithms (DSA)

Credits: 05

INDEX

S.No	Title	Page No	Remarks
1)	WEEK 1: 1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.	4-11	
2)	WEEK 2: 2a. Write a python program to perform linear search on the give input. 2b. Write a python program to perform binary search on the given. Verify by taking sorted array as an input. 2c. Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.	11-16	
3)	WEEK 3: 3a. Write a program to perform Bubble Sort technique on the user given input. 3b. Write a program to perform Insertion Sort technique on the user given input. 3c. Write a program to perform Selection Sort technique on the user given input. 3d. Write a program to perform Merge Sort technique on the user given input. 3e. Write a program to perform Quick Sort technique on the user given input.	16-25	
4)	WEEK 4: 4a. Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user. a. Create a linked list. b. Insert at beginning. c. Insert at end. d. Insert at a specific index. e. Delete by value. f. Delete 1st node. g. Delete last node. h. Count no. of nodes i. Display. j. Exit.	26-66	

	4b. Write a python program to implement a double linked list which would perform the below operations, by taking an input from the user. 4c. Write a python program to implement a circular linked list which would perform the below operations, by taking an input from the user. Also add these two options accordingly. 1. Display head and tail. 2. Print data from tail to head.		
5)	WEEK 5: Write a python program to implement a stack using an a)Array, b)Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.	66-74	
6)	WEEK 6: To implement Queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu driven approach. 6a. Using Array, Linked list, Data structures. 6b. Circular Queue.	74-86	
7)	WEEK 7: Write a python program to implement binary tree 7a. Using Array, and 7b. Linked List With all of its operation.	86-100	
8)	WEEK 8: Write a python program to implement Binary Search tree with its operation.	100-119	
9)	WEEK-9: To implement hashing using separate chaining, linear and quadratic probing(using a)array and b)linked list)	120-131	

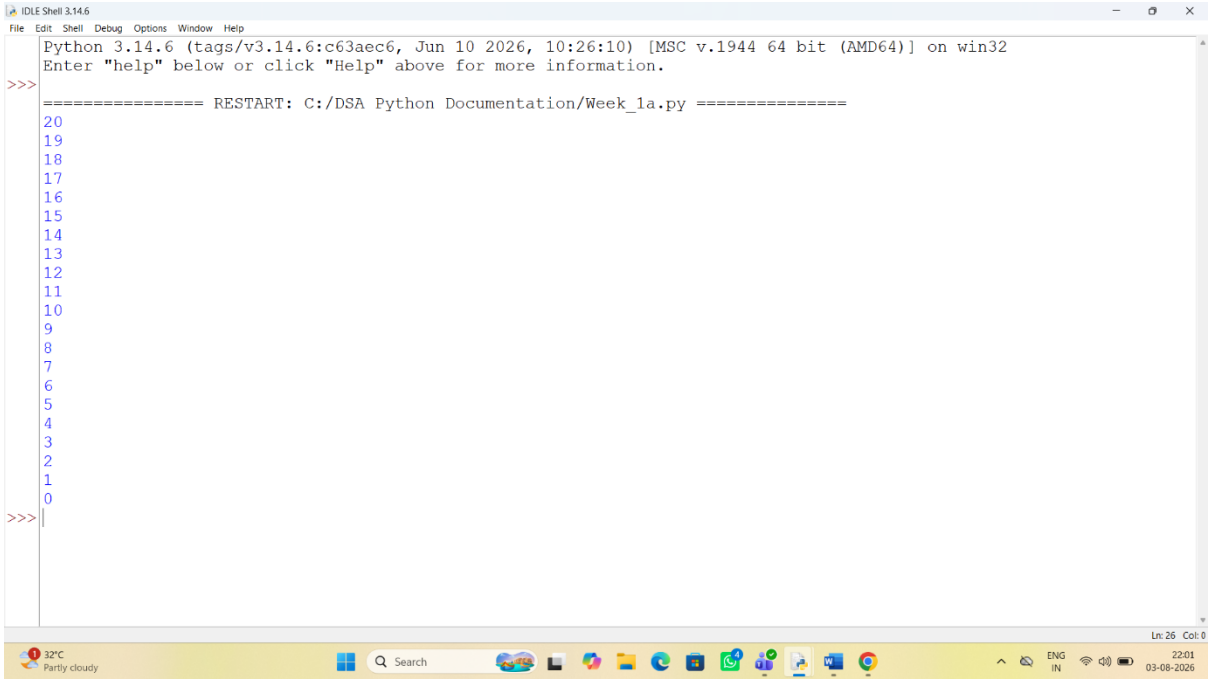
Week-1

1a)Aim: To implement recursion in Python by creating a function that prints numbers in descending order from a given number.

Program:

```
def launch(n):
    print(n)
    if n>0:
        n-=1
        return launch(n)
result=launch(20)
```

Output:



Explanation:

- The function launch(n) prints the current value of n.
- If n is greater than 0, it decreases n by 1 and calls itself recursively.

- The recursion continues until n becomes 0, which is the base case, and the function stops.

Error Table:

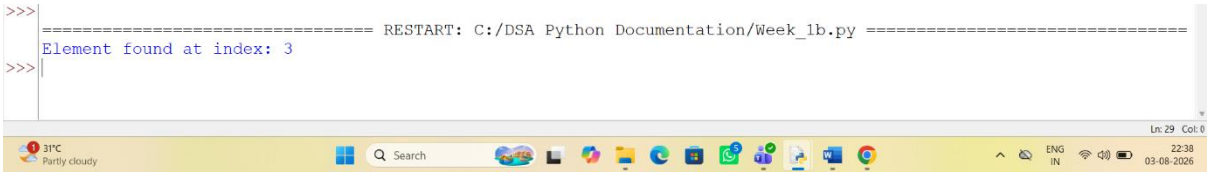
S.No	Error Obtained	Rectification
01.	Didn't write the print statement after definition.	After execution I found the mistake and added it.

1b)Aim: To implement a recursive binary search in Python to find the position (index) of a given element in a list. The program recursively checks each element of the list until the target element is found or the end of the list is reached.

Program:

```
def empid_Search(arr,target,n=0):
    if n==len(arr):
        return -1
    if arr[n]==target:
        return n
    return empid_Search(arr,target,n+1)
arr=[10,20,30,50,607,809,506]
result=empid_Search(arr,50)
if result!=-1:
    print("Element found at index:",result)
else:
    print("Element not found")
```

Output:



Explanation:

This program uses recursion to perform a linear search for an employee ID (or any target element) in a list.

1. The function empid_Search(arr, target, n=0) is defined with three parameters:
    - arr: The list in which the search is performed.
    - target: The element to be searched.
    - n: The current index, initialized to 0.
2. The statement if n == len(arr): return -1 is the base condition. It checks whether all elements in the list have been examined. If the end of the list is reached without finding the target, the function returns -1.
3. The statement if arr[n] == target: return n compares the current element with the target. If they are equal, the function returns the current index where the target is found.
4. If the current element does not match the target, the function calls itself recursively with the next index using return empid_Search(arr, target, n + 1).
5. The list arr = [10, 20, 30, 50, 607, 809, 506] is created, and the function is called to search for the element 50.
6. The returned value is stored in the variable result.
7. If result is not equal to -1, the program prints the index where the element is found. Otherwise, it prints "Element not found."

Error table:

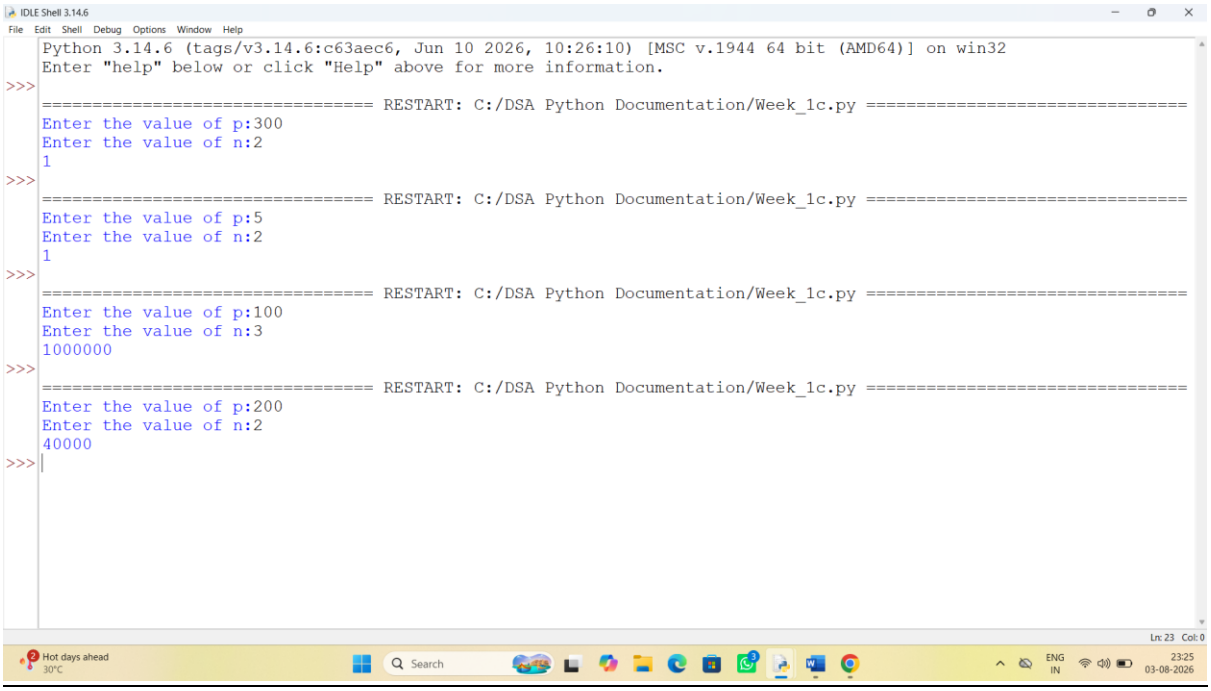
S.No	Error Obtained	Rectification
01.	Recursion Error: maximum recursion depth exceeded	Increment the index in the recursive call: return empid_Search(arr, target, n + 1).

1c)Aim: This Python program uses recursion to calculate the value of p^n (p raised to the power n).

Program:

```
def power(p,n):
    if n==0:
        return 1
    else:
        return p*power(p,n-1)
p=int(input("Enter the value of p:"))
n=int(input("Enter the value of n:"))
result=power(p,n)
print(result)
```

Output:



Explanation:

 The program is used to find the power of a number (p^n) using recursion.
 It takes two inputs from the user:

- p → Base number
- n → Exponent (power)

 The function power(p, n) is called to calculate the result.
 If the exponent n is **0**, the function returns **1** because any number raised to the power 0 is
 If n is not 0, the function calls itself with n - 1 and multiplies the result by p.
 This recursive process continues until the exponent becomes **0**.
 After reaching the base case, each recursive call returns its result, and the final value of p^n is obtained.
 The result is stored in the variable result.

 Finally, the program prints the calculated power of the given number.

Error Table:

S.No	Error Obtained	Rectification
01.	Not a int value of n we can also take float in place of int, I have taken float value to check.	By replacing int by float.

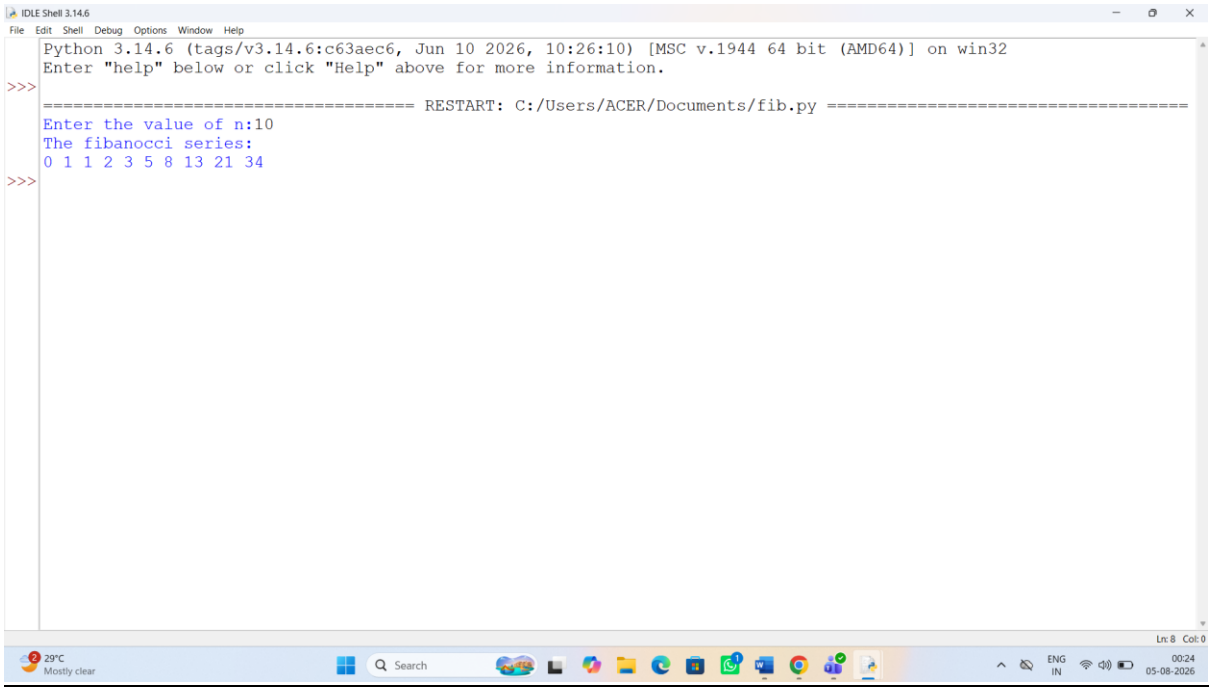
1d)Aim: To write a Python program that generates and prints the Fibonacci series up to n terms using recursion**.**

Program:

```
def fib(n):
   if n<=1:
      return n
   return fib(n-1)+ fib(n-2)
n=int(input("Enter the value of n:"))
print("The fibanocci series:")
for i in range (n):
```

print(fib(i),end=" ")

Output:



Explanation:

- The program uses a recursive function fib(n) to generate Fibonacci numbers.
- The base condition returns 0 when n = 0 and 1 when n = 1.
- For values greater than 1, the function returns the sum of the previous two Fibonacci numbers: fib(n-1) + fib(n-2).
- The user enters the number of terms (n) to be displayed.
- A for loop calls the fib() function repeatedly for each term from 0 to n-1.
- Finally, the program prints the Fibonacci series in sequence.

Error Table:

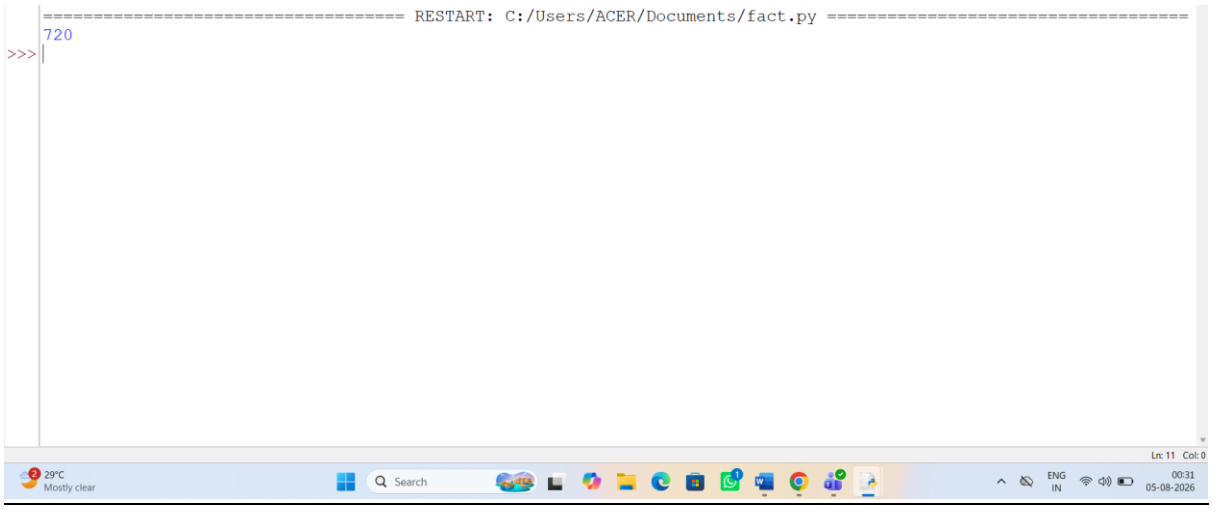
S.No	Error Obtained	Rectification
01.	No Errors obtained	----------

1e)Aim: To write a Python program to find the factorial of a given number using recursion.

Program:

```
def fact(n):
    if n<=1:
        return n
    return n*fact(n-1)
print(fact(6))
```

Output:



Explanation:

- The program defines a recursive function fact(n) to calculate the factorial of a number.
- The base condition checks if n <= 1. If true, it returns 1 (for factorial, 0! = 1 and 1! = 1).

 Otherwise, the function returns n * fact(n-1), multiplying the current number by the factorial of the previous number.
 The function is called with the value 6.

 Finally, the program prints the factorial of 6, which is 720.

Error Table:

S.No	Error Obtained	Rectification
01.	No Errors obatined	----------

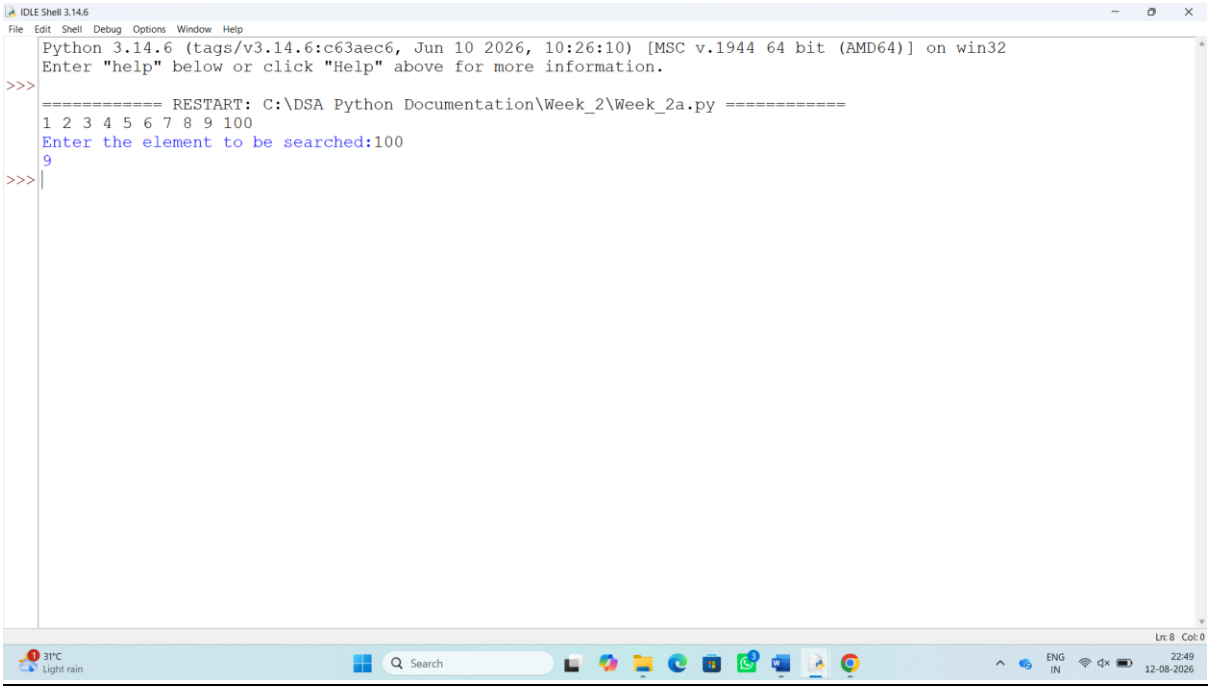
Week-2

2a)Aim: To write a Python program to search for a given element in an array using the Linear Search technique and display its position.

Program:

```
def linear_search(arr,key):
    for i in range(len(arr)):
        if arr[i]==key:
            return i
        else:
            i=i+1
    return 0
arr=list(map(int,input().split()))
key=int(input("Enter the element to be searched:"))
print(linear_search(arr,key))
```

Output:



Explanation:

- The program defines a function linear_search() to search for an element in a list.
- It takes two inputs: the array (arr) and the element to be searched (key).
- The for loop checks each element of the array one by one.
- If arr[i] == key, the function returns the index of the element.
- If the element is not found, the function returns 0.
- The user enters the array elements and the element to be searched.

- Finally, the program calls linear_search() and prints the result.

Error Table:

S.No	Error Obtained	Rectification
01.	Error in input taking of array	Rectified by split.

2b)Aim:To write a Python program to search for a given element in an array using the recursive Binary Search technique, by checking whether the array is sorted and sorting it if necessary.

Program:

FOR SORTED ARRAY:

```
def binary_search(arr,key,low,high):
    if low>high:
        return -1
    mid=(low+high)//2
    if arr[mid]==key:
        return mid
    elif arr[mid]>key:
        return binary_search(arr,key,low,mid)
    else:
        return binary_search(arr,key,mid+1,high)
    return 0
arr=list(map(int,input().split()))
key=int(input("Enter the element to be searched:"))
print(binary_search(arr,key,0,len(arr)))
```

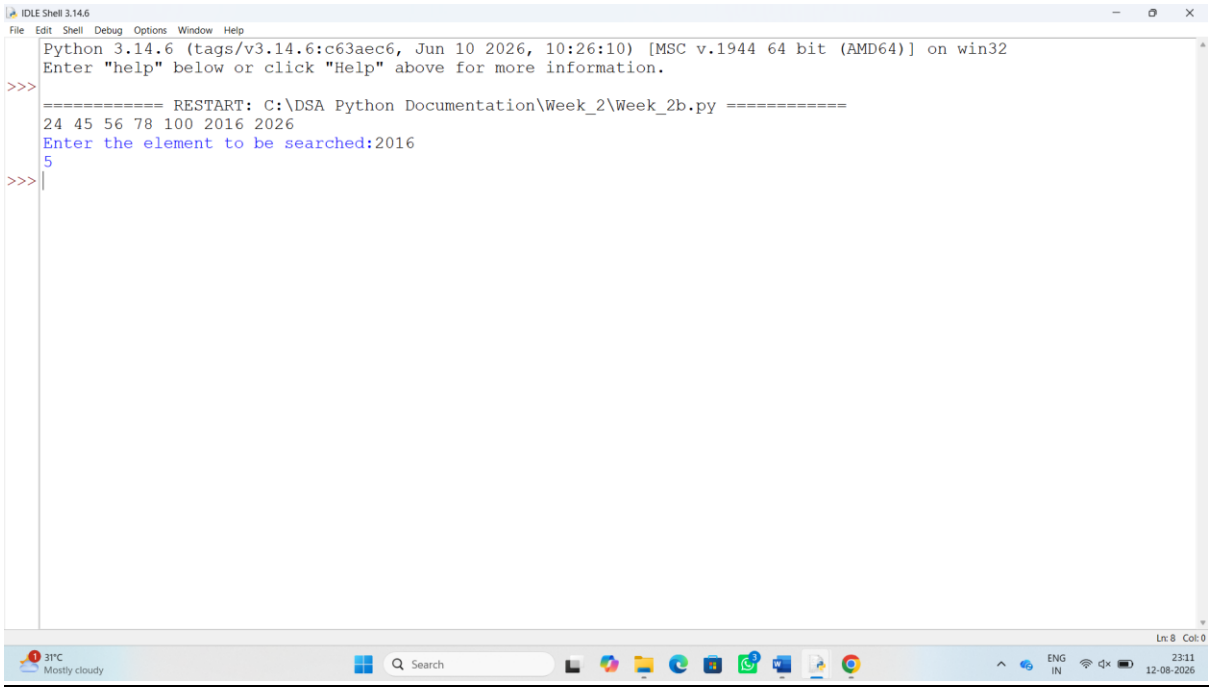
FOR UNSORTED ARRAY:

```
def binary_search(arr,key,low,high):
    if low>high:
        return -1
    mid=(low+high)//2
    if arr[mid]==key:
        return mid
    elif arr[mid]>key:
```

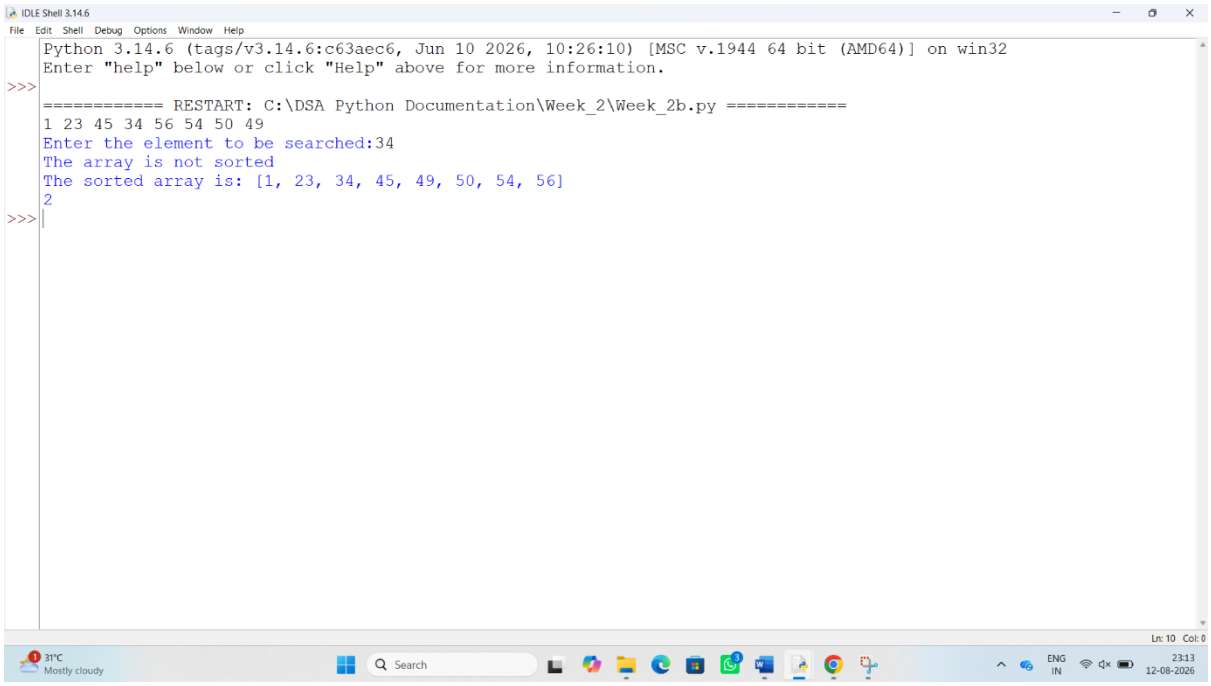
```
        return binary_search(arr,key,low,mid-1)
    else:
        return binary_search(arr,key,mid+1,high)
    return 0
arr=list(map(int,input().split()))
key=int(input("Enter the element to be searched:"))
if arr==sorted(arr):
    print("The array is sorted")
else:
    print("The array is not sorted")
    arr.sort()
print("The sorted array is:",arr)
print(binary_search(arr,key,0,len(arr)))
```

Output:

For Sorted array



For Unsorted array:



Explanation:

- The program uses Binary Search to find a given element in an array.
- The binary_search() function works recursively by dividing the search range into two halves.
- It calculates the middle position using (low + high) // 2.
- If the middle element is equal to the key, its index is returned.

 If the middle element is greater than the key, the function searches the left half.
 If the middle element is smaller than the key, it searches the right half.
 For an unsorted array, the program first checks whether the array is sorted using arr == sorted(arr).
 If it is not sorted, arr.sort() is used to arrange the elements in ascending order.

 Finally, the Binary Search function is called to search for the required element.

Error Table:

S.No	Error Obtained	Rectification
01.	Error in if clause statement	Rectified by keeping ==

Week-3

3a)Aim: To write a Python program to sort the elements of an array in ascending order using the Bubble Sort technique**.**

Program:

```
def bubble_sort(arr):
    n=len(arr)
    for i in range (n):
        for j in range(0,n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]
n=int(input("enter the number of elements to be sorted:" ))
```

```
arr=list(map(int,input("enter the elements into the array").split()))

print("sorted array")

bubble_sort(arr)

for i in arr:

    print(i,end="  ")
```

Output:



Explanation:

- def bubble_sort(arr): → Defines a function named bubble_sort to sort the array.
- n = len(arr) → Finds the number of elements in the array.
- for i in range(n): → Repeats the sorting process for multiple passes.
- for j in range(0, n-i-1): → Compares adjacent elements in the array.
- if arr[j] > arr[j+1]: → Checks whether the current element is greater than the next element.
- arr[j], arr[j+1] = arr[j+1], arr[j] → Swaps the two elements if they are in the wrong order.
- After each pass, the largest unsorted element moves to the end of the array.
- return arr → Returns the sorted array.

- numbers = [64, 34, 25, 12, 22, 11, 90] → Creates an array containing the elements to be sorted.
- print(bubble_sort(numbers)) → Calls the function and displays the sorted array.
- The final output is in ascending order:
  [11, 12, 22, 25, 34, 64, 90].

Error Table:

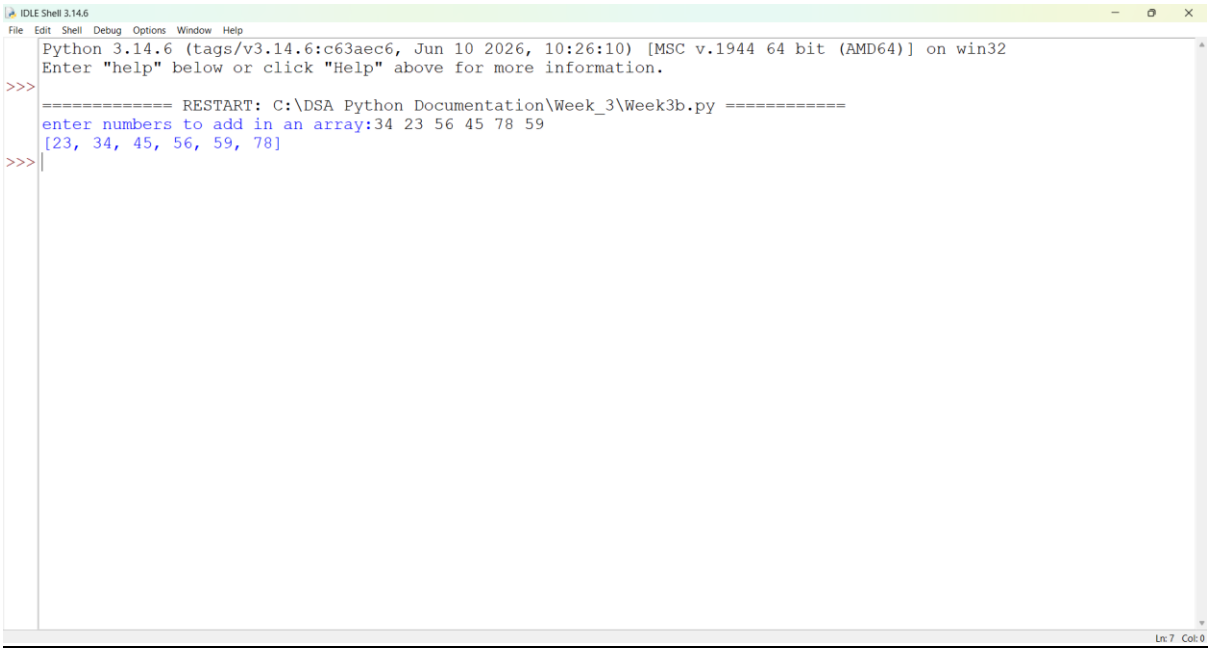
S.no	Error	Explanation
1.	No errors	----------

3b)Aim: To write a Python program to sort the elements of an array in ascending order using the Insertion Sort technique.

Program:

```
def insertion_sort(arr):
    n=len(arr)
    for i in range(1,n):
        key=arr[i]
        j=i-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j=j-1
        arr[j+1]=key
    return arr
arr=list(map(int,input("enter numbers to add in an array:").split()))
print(insertion_sort(arr))
```

Output:



Explanation:

- Insertion Sort is a sorting technique that builds the sorted array one element at a time.
- It works similar to the way we arrange playing cards in our hand.
- The first element is considered already sorted.
- The next element is selected and called the key.
- The key is compared with the elements on its left side.
- If an element on the left is greater than the key, it is shifted one position to the right.
- This comparison and shifting continues until the correct position for the key is found.
- The key is then inserted into its correct position.
- The same process is repeated for all remaining elements.

- At the end, the complete array is sorted in ascending order.

Error Table:

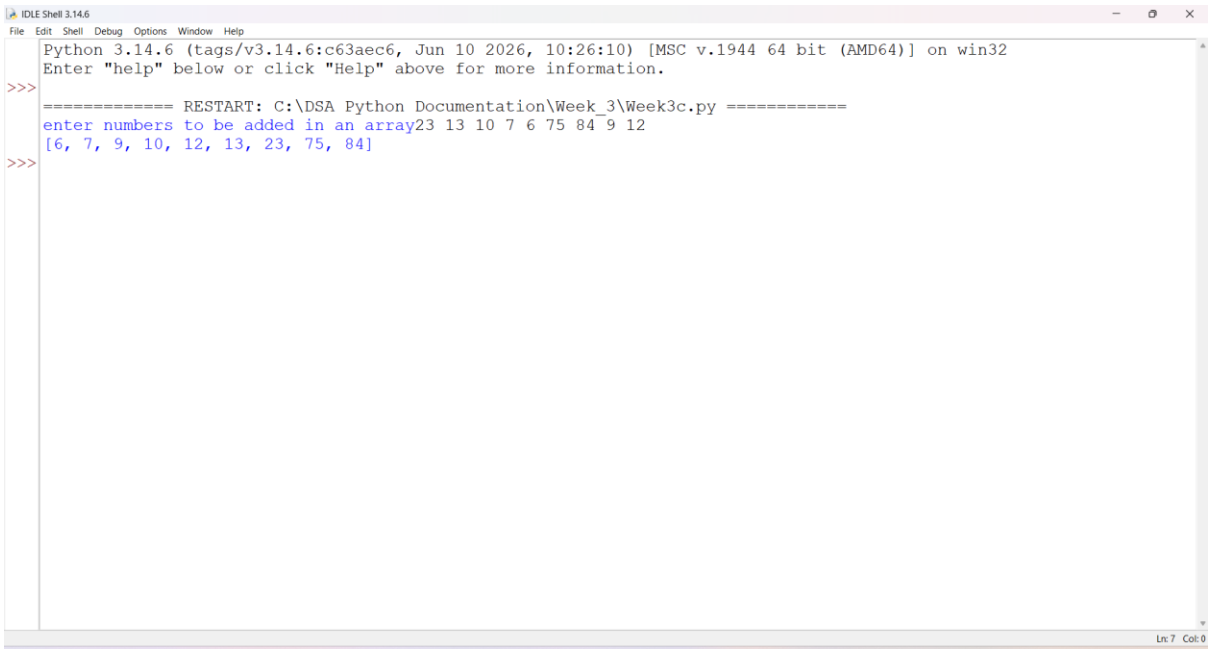
S.No	Error Obtained	Explanation
1)	No errors	----------

3c)Aim: To write a program to sort a given array of elements in ascending order using the Selection Sort technique.

Program:

```
def selection_sort(arr):
    n=len(arr)
    for i in range (n-10):
        min_index=i
        for j in range (i+1,n):
            if arr[j]<arr[min_index]:
                min_index=j
            arr[i],arr[min_index]=arr[min_index], arr[i]
        return arr
arr=list(map(int,input("enter numbers to be added in an array").split()))
print(selection_Sort(arr))
```

Output:



Explanation:

 Selection Sort is a simple sorting algorithm used to arrange elements in ascending or descending order.
 It divides the array into two parts:

- Sorted part
- Unsorted part

 In each pass, it finds the smallest element from the unsorted part.
 The smallest element is then swapped with the first element of the unsorted part.
 After every pass, the sorted part becomes one element larger.
 This process continues until all elements are sorted.
 Selection Sort performs sorting in-place, so it does not require an extra array.
 The time complexity of Selection Sort is $O(n^2)$.

 It is easy to understand and implement, but it is not efficient for large datasets.

Error Table:

S.No	Errors obtained	Explanation
1.	Error in function call.	Rectified it by proper function call.
2.	Error in for loop	Rectified it by keeping n-1.

3d)Aim: To write a program to sort a given array of elements in ascending order using the Merge Sort algorithm.

Program

```
def merge_sort(arr):
    if len(arr)<=1:
        return arr
    mid=len(arr)//2
    left=arr[:mid]
```

```
    right=arr[mid:]
    merge_sort(left)
    merge_sort(right)
    i=j=k=0
    while i<len(left) and j<len(right):
            if left[i]<=right[j]:
                arr[k]=left[i]
                i+=1
            else:
                arr[k]=right[j]
                j+=1
     k+=1
   while i<len(left):
        arr[k]=left[i]
        i+=1
        k+=1
        while j<len(right):
           arr[k]=right[j]
           j+=1
        k+=1
n=int(input("enter the number of elements to sort"))
arr=list(map(int,input("enter the elements").split()))
merge_sort(arr)
```

```
print("sorted array")

for elements in arr:

    print(elements,end=" ")
```

Output:



Explanation:

- Merge Sort is a sorting algorithm based on the Divide and Conquer technique.
- It divides the array into two smaller parts repeatedly.
- This division continues until each part contains only one element.
- A single element is considered already sorted.
- The smaller parts are then merged together in the correct order.
- During merging, the elements of the two parts are compared one by one.
- The smaller element is placed first in the merged array.
- This merging process continues until the entire array is sorted.
- Merge Sort has a time complexity of O(n log n).

- It requires extra memory for the merging process.

Error Table:

S.No	Errors Obtained	Explanation
1.	No errors.	----------

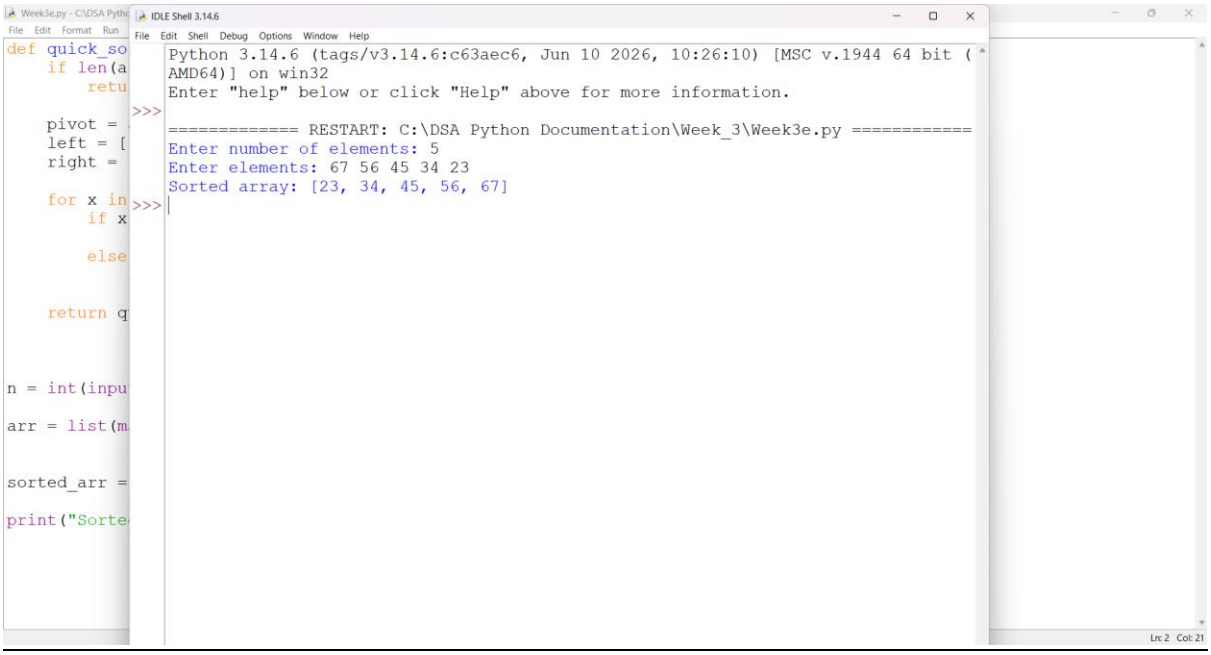
3e)Aim: To write a program to sort a given array of elements in ascending order using the Quick Sort algorithm based on the Divide and Conquer technique.

Program:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]
    left = []
    right = []
    for x in arr[:-1]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)
   return quick_sort(left) + [pivot] + quick_sort(right)
n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter elements: ").split()))
sorted_arr = quick_sort(arr)
print("Sorted array:", sorted_arr)
```

Output:



Explanation:

- Quick Sort is a sorting algorithm based on the Divide and Conquer technique.
- It selects one element from the array as a pivot.
- The elements are divided into two parts based on the pivot:

  - Elements smaller than the pivot are placed on the left.
  - Elements greater than the pivot are placed on the right.

- The pivot is now in its correct position in the sorted array.
- The same process is repeated for the left and right sub-arrays.
- This process continues until the sub-arrays contain one or no elements.
- Finally, all the elements are arranged in ascending order.
- Quick Sort has an average time complexity of O(n log n).

- Its worst-case time complexity is $O(n^2)$, depending on the choice of pivot.

Errors Table:

S.No	Errors obtained	Explanation
1.	No errors	----------

WEEK – 4

4a. AIM: Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user.

a. Create a linked list. b. Insert at beginning. c. Insert at end.

d. Insert at a specific index. e. Delete by value. f. Delete 1st node.

g. Delete last node. h. Count no. of nodes i. Display.

j. Exit.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # a. Create a linked list
    def create(self):
        n = int(input("Enter the number of nodes: "))

        for i in range(n):
            data = int(input("Enter the data: "))
            self.insert_end(data)
```

```
    print("Linked list created successfully.")

# b. Insert at beginning
def insert_beginning(self):
    data = int(input("Enter the data: "))

    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node

    print("Node inserted at beginning.")

# c. Insert at end
def insert_end(self, data=None):
    if data is None:
        data = int(input("Enter the data: "))

    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        return

    temp = self.head

    while temp.next is not None:
```

```
            temp = temp.next

        temp.next = new_node

        if data is not None:
            print("Node inserted at end.")

    # d. Insert at a specific index
    def insert_at_index(self):
        index = int(input("Enter the index: "))
        data = int(input("Enter the data: "))

        if index < 0:
            print("Invalid index.")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node
            print("Node inserted at index", index)
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
```

```
            print("Index out of range.")
            return
        temp = temp.next

    if temp is None:
        print("Index out of range.")
        return

    new_node = Node(data)
    new_node.next = temp.next
    temp.next = new_node

    print("Node inserted at index", index)

# e. Delete by value
def delete_by_value(self):
    value = int(input("Enter the value to delete: "))

    if self.head is None:
        print("List is empty.")
        return

    if self.head.data == value:
        self.head = self.head.next
        print("Node deleted successfully.")
        return
```

```
    temp = self.head

    while temp.next is not None:
        if temp.next.data == value:
            temp.next = temp.next.next
            print("Node deleted successfully.")
            return
        temp = temp.next

    print("Value not found.")

# f. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return

    self.head = self.head.next
    print("First node deleted.")

# g. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty.")
        return

    if self.head.next is None:
```

```
            self.head = None
            print("Last node deleted.")
            return

        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        temp.next = None
        print("Last node deleted.")

    # h. Count number of nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    # i. Display
    def display(self):
        if self.head is None:
            print("List is empty.")
```

```
            return

        temp = self.head

        print("Linked List:", end=" ")

        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


# Main program
linked_list = SinglyLinkedList()

while True:
    print("\n----- SINGLY LINKED LIST MENU -----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display")
```

```
print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    linked_list.create()

elif choice == 2:
    linked_list.insert_beginning()

elif choice == 3:
    linked_list.insert_end()

elif choice == 4:
    linked_list.insert_at_index()

elif choice == 5:
    linked_list.delete_by_value()

elif choice == 6:
    linked_list.delete_first()

elif choice == 7:
    linked_list.delete_last()

elif choice == 8:
    linked_list.count_nodes()
```

```
    elif choice == 9:
        linked_list.display()

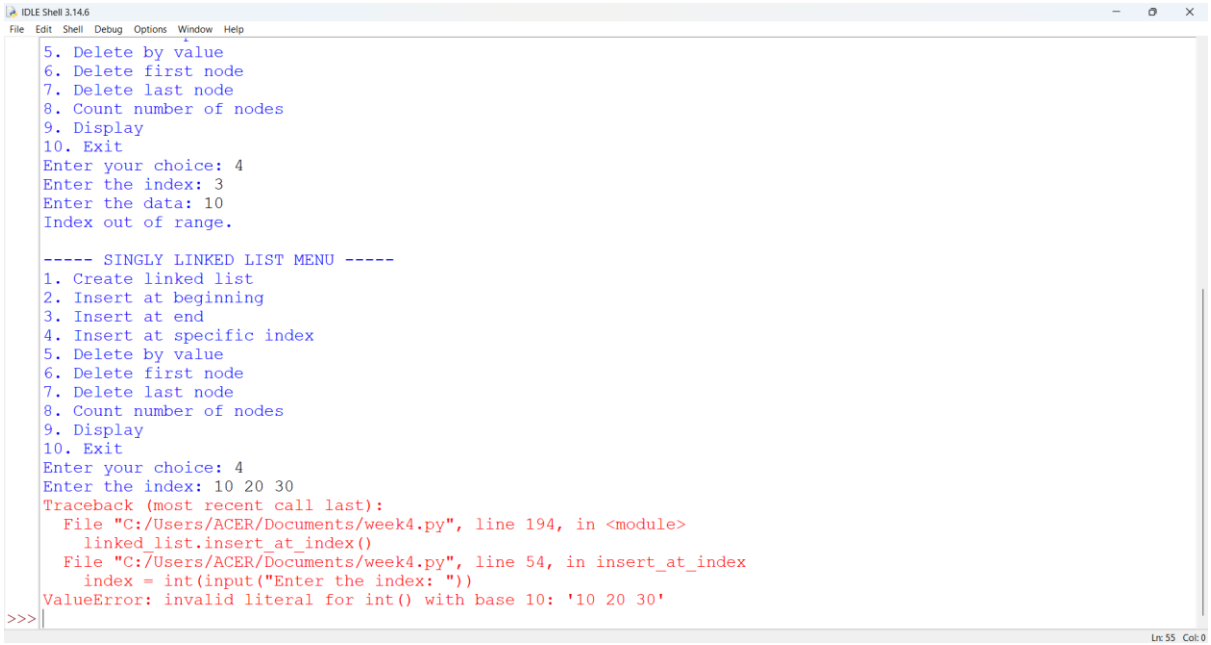
    elif choice == 10:
        print("Exiting program...")
        break

    else:
        print("Invalid choice. Please try again.")
```

Output:







EXPLANATION:

◈ Class Definitions

1. Node **class**
    - Represents a single node in the linked list.
    - Each node has:
        - data → stores the value.
        - next → pointer to the next node (initially None).

2. SLL **class**
    - Manages the linked list operations.
    - Has a head pointer (initially None) that points to the first node.

◈ Operations in SLL

1. Create Linked List

- Asks user for number of nodes.
- For each node:
    - Creates a new Node.
    - If list is empty → new node becomes head.
    - Otherwise → traverses to the end and attaches the new node.

2. Insert at Beginning

- Creates a new node.
- Points its next to current head.
- Updates head to new node.

3. Insert at End

- Creates a new node.
- If list is empty → new node becomes head.
- Otherwise → traverses to last node and attaches new node.

4. Insert at Index

- Asks for index and data.
- If index = 0 → behaves like "insert at beginning".
- Otherwise → traverses to (index-1) node and inserts new node in between.

5. Delete from Beginning

- Moves head to head.next.
- Effectively removes the first node.

6. Delete from End

- If list empty → prints "List is empty".
- If only one node → sets head = None.
- Otherwise → traverses to second-last node and sets its next = None.

7. Delete from Index

- Asks for index.
- If index = 0 → removes head.
- Otherwise → traverses to (index-1) node and skips the next node.

8. Count Nodes

- Traverses through list.
- Counts nodes until None.

9. Display

- Traverses from head to end.
- Prints each node's data in data -> data -> None format.

◈ Main Program

- Runs an infinite loop with a menu:
  - Options 1–9 → perform linked list operations.
  - Option 10 → exits program.
- Uses if-elif to call respective methods.

Error Table:

S.NO	Error Obtained	Rectification
1.	Index out of range When inserting/deleting at an index greater than the number of nodes.	Traverse carefully; if temp becomes None before reaching index, print "Index out of range" and stop.
2.	Indentation Error In the Count node part	It was rectified later, By maintaining proper indentation.

4b. AIM: Write a python program to implement a double linked list which would perform the below operations, by taking an input from the user.

a. Create a linked list. b. Insert at beginning. c. Insert at end.

d. Insert at a specific index. e. Delete by value. f. Delete 1st node.

g. Delete last node. h. Count no. of nodes i. Display. J.exit.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # a. Create a linked list
    def create(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
```

```
            temp = self.head
            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        print("Node created successfully")

    # b. Insert at beginning
    def insert_beginning(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        self.head = new_node
        print("Node inserted at beginning")

    # c. Insert at end
    def insert_end(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is None:
```

```
            self.head = new_node
        else:
            temp = self.head
            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        print("Node inserted at end")

    # d. Insert at a specific index
    def insert_index(self):
        data = int(input("Enter the data: "))
        index = int(input("Enter the index: "))

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head

            if self.head is not None:
                self.head.prev = new_node
```

```
            self.head = new_node
            print("Node inserted successfully")
            return

        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next

        if temp is None:
            print("Invalid index")
            return

        new_node = Node(data)
        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

        print("Node inserted successfully")

    # e. Delete by value
```

```
def delete_value(self):
    value = int(input("Enter the value to delete: "))
    temp = self.head

    while temp is not None:
        if temp.data == value:

            if temp.prev is not None:
                temp.prev.next = temp.next
            else:
                self.head = temp.next

            if temp.next is not None:
                temp.next.prev = temp.prev

            print("Node deleted successfully")
            return

        temp = temp.next

    print("Value not found")

# f. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty")
        return
```

```
        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("First node deleted")

    # g. Delete last node
    def delete_last(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp.next is not None:
            temp = temp.next

        if temp.prev is None:
            self.head = None
        else:
            temp.prev.next = None

        print("Last node deleted")

    # h. Count number of nodes
```

```
def count(self):
    temp = self.head
    count = 0

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

# i. Display
def display(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    print("Doubly Linked List:")

    while temp is not None:
        print(temp.data, end=" <-> ")
        temp = temp.next

    print("None")
```

```
# Main program
dll = DoublyLinkedList()

while True:
    print("\n--- DOUBLY LINKED LIST MENU ---")
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        dll.insert_beginning()

    elif choice == 3:
        dll.insert_end()
```

```
    elif choice == 4:
        dll.insert_index()

    elif choice == 5:
        dll.delete_value()

    elif choice == 6:
        dll.delete_first()

    elif choice == 7:
        dll.delete_last()

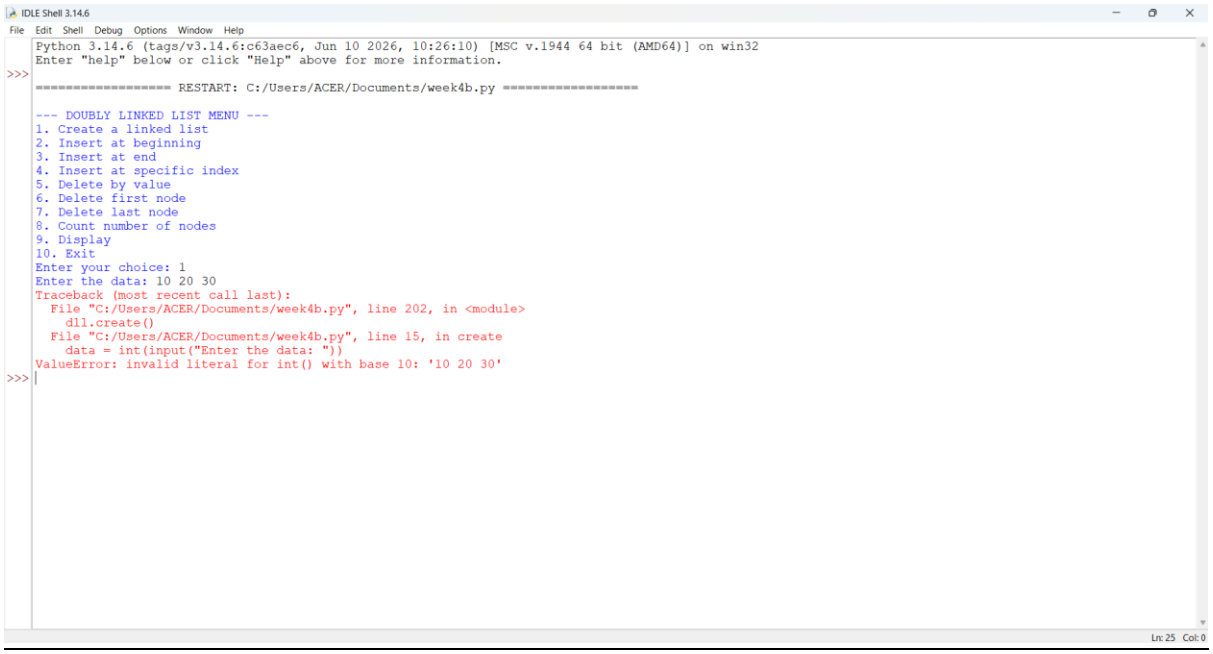
    elif choice == 8:
        dll.count()

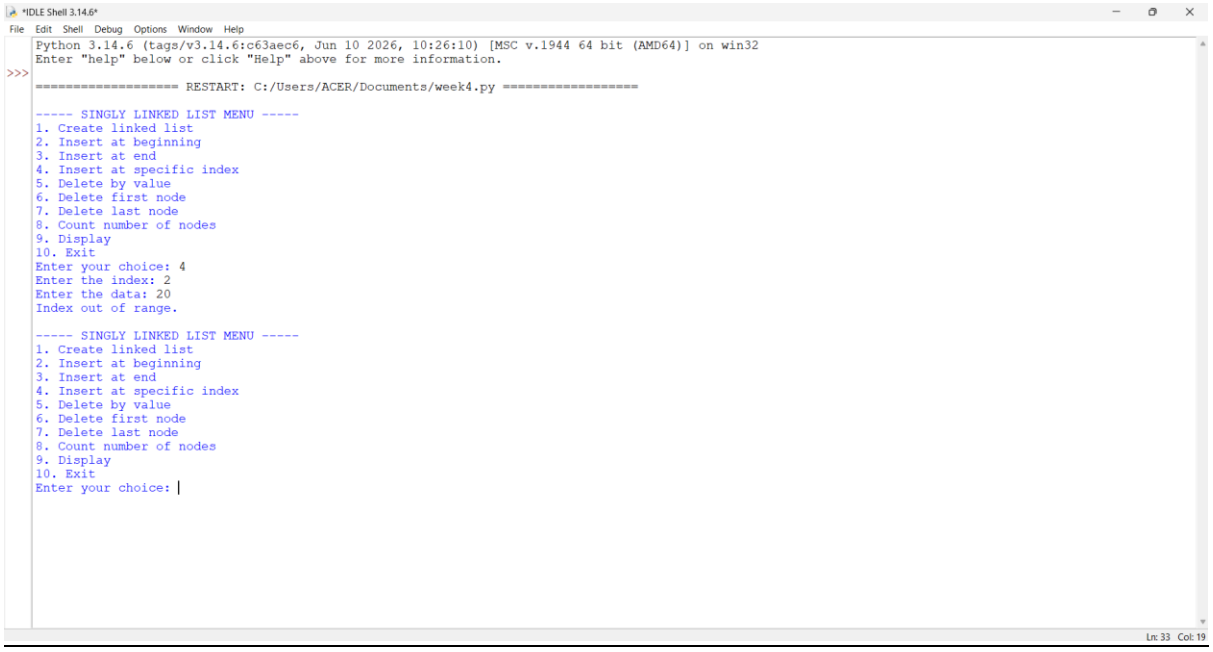
    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

Output:


IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ACER/Documents/week4b.py ==========
--- DOUBLY LINKED LIST MENU ---
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter the data: 10 20 30
Traceback (most recent call last):
File "C:/Users/ACER/Documents/week4b.py", line 202, in <module>
dll.create()
File "C:/Users/ACER/Documents/week4b.py", line 15, in create
data = int(input("Enter the data: "))
ValueError: invalid literal for int() with base 10: '10 20 30'
>>>
Ln: 25 Col: 0


IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ACER/Documents/week4.py ==========
----- SINGLY LINKED LIST MENU -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter the index: 2
Enter the data: 20
Index out of range.
----- SINGLY LINKED LIST MENU -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice:
Ln: 33 Col: 19



EXPLANATION:

1. Node Class

- Each node stores:
  - data: the actual value.
  - prev: pointer to the previous node.
  - next: pointer to the next node.
- This structure allows traversal in both directions (forward and backward).

2. Double Linked List Class

This class manages the linked list. It starts with self.head = None (empty list). It provides several operations:

a. Create Linked List

- User specifies how many nodes to create.
- Each new node is linked at the end.
- prev pointer connects the new node to the previous one.

b. Insert at Beginning

- A new node is created.
- Its next points to the current head.

- The old head’s prev points back to the new node.
- The new node becomes the head.

c. Insert at End

- Traverse to the last node.
- Attach the new node at the end.
- Update prev of the new node to point to the last node.

d. Insert at Specific Index

- Traverse until the given index.
- Insert the new node between two nodes.
- Adjust both next and prev pointers to maintain the chain.

e. Delete by Value

- Search for the node with the given value.
- If found:
    - If it’s the head, move head to the next node.
    - Otherwise, adjust prev.next and next.prev to bypass the node.
- This removes the node from the chain.

f. Delete First Node

- Move head to the second node.
- Set its prev to None.

g. Delete Last Node

- Traverse to the last node.
- Remove it by setting its previous node’s next to None.

h. Count Nodes

- Traverse the list, incrementing a counter.
- Prints the total number of nodes.

i. Display

- Traverse from head to end.

- Print each node's data in the format: data <-> data <-> ... None

3. Main Program (Menu-Driven)

- Runs an infinite loop with options:
    1. Create Linked List
    2. Insert at Beginning
    3. Insert at End
    4. Insert at Specific Index
    5. Delete by Value
    6. Delete First Node
    7. Delete Last Node
    8. Count Nodes
    9. Display
    10. Exit

Based on user choice, the corresponding method is executed

Error table:

S.NO	Error Obtained	Rectification
1.	Invalid index entered (negative or too large).	Add a check: if index < 0 → print "Invalid index"; if traversal reaches None before index → print "Index out of range".
2.	Deleting from an empty list.	Before deletion, check if self.head is None: → print "Doubly Linked List is empty."
3.	Value not found during deletion by value.	Traverse list; if node not found → print "value not found in the Linked List."
4.	Insertion at index when list is empty but index > 0.	Handle separately: if self.head is None and index! = 0 → print "Index out of range."

4c. AIM: Write a python program to implement a circular linked list which would perform the below operations, by taking an input from the user.

a. Create a linked list. b. Insert at beginning. c. Insert at end.

d. Insert at a specific index. e. Delete by value. f. Delete 1st node.

g. Delete last node. h. Count no. of nodes i. Display.

j. Display head and tail. k. Print data from tail to head. l. Exit.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # a. Create a linked list
    def create(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
```

```
        else:
            new_node.next = self.head
            self.tail.next = new_node
            self.tail = new_node

        print("Node created successfully")

    # b. Insert at beginning
    def insert_beginning(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.head = new_node
            self.tail.next = self.head

        print("Node inserted at beginning")

    # c. Insert at end
    def insert_end(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)
```

```
    if self.head is None:
        self.head = new_node
        self.tail = new_node
        new_node.next = self.head
    else:
        new_node.next = self.head
        self.tail.next = new_node
        self.tail = new_node

    print("Node inserted at end")

# d. Insert at a specific index
def insert_index(self):
    data = int(input("Enter the data: "))
    index = int(input("Enter the index: "))

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
```

```
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.head = new_node
            self.tail.next = self.head

        print("Node inserted successfully")
        return

    if self.head is None:
        print("Invalid index")
        return

    temp = self.head

    for i in range(index - 1):
        temp = temp.next

        if temp == self.head:
            print("Invalid index")
            return

    new_node = Node(data)
    new_node.next = temp.next
    temp.next = new_node

    if temp == self.tail:
```

```
            self.tail = new_node

        print("Node inserted successfully")

    # e. Delete by value
    def delete_value(self):
        value = int(input("Enter the value to delete: "))

        if self.head is None:
            print("List is empty")
            return

        current = self.head
        previous = self.tail

        while True:
            if current.data == value:

                if self.head == self.tail:
                    self.head = None
                    self.tail = None

                else:
                    previous.next = current.next

                    if current == self.head:
                        self.head = current.next
```

```
                if current == self.tail:
                    self.tail = previous

                self.tail.next = self.head

                print("Node deleted successfully")
                return

            previous = current
            current = current.next

            if current == self.head:
                break

        print("Value not found")

    # f. Delete first node
    def delete_first(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head == self.tail:
            self.head = None
            self.tail = None
        else:
```

```
        self.head = self.head.next
        self.tail.next = self.head

    print("First node deleted")

# g. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head == self.tail:
        self.head = None
        self.tail = None
    else:
        temp = self.head

        while temp.next != self.tail:
            temp = temp.next

        temp.next = self.head
        self.tail = temp

    print("Last node deleted")

# h. Count number of nodes
def count(self):
```

```
    if self.head is None:
        print("Number of nodes: 0")
        return

    count = 0
    temp = self.head

    while True:
        count += 1
        temp = temp.next

        if temp == self.head:
            break

    print("Number of nodes:", count)

# i. Display
def display(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    print("Circular Linked List:")

    while True:
```

```
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("HEAD")

    # j. Display head and tail
    def display_head_tail(self):
        if self.head is None:
            print("List is empty")
            return

        print("Head:", self.head.data)
        print("Tail:", self.tail.data)

    # k. Print data from tail to head
    def print_tail_to_head(self):
        if self.head is None:
            print("List is empty")
            return

        values = []
        temp = self.head

        while True:
```

```
            values.append(temp.data)
            temp = temp.next

            if temp == self.head:
                break

        print("Data from tail to head:")

        for data in reversed(values):
            print(data, end=" -> ")

        print("HEAD")


# Main program
cll = CircularLinkedList()

while True:
    print("\n--- CIRCULAR LINKED LIST MENU ---")
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
```

```
print("9. Display")
print("10. Display head and tail")
print("11. Print data from tail to head")
print("12. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    cll.create()

elif choice == 2:
    cll.insert_beginning()

elif choice == 3:
    cll.insert_end()

elif choice == 4:
    cll.insert_index()

elif choice == 5:
    cll.delete_value()

elif choice == 6:
    cll.delete_first()

elif choice == 7:
    cll.delete_last()
```

```
    elif choice == 8:
        cll.count()

    elif choice == 9:
        cll.display()

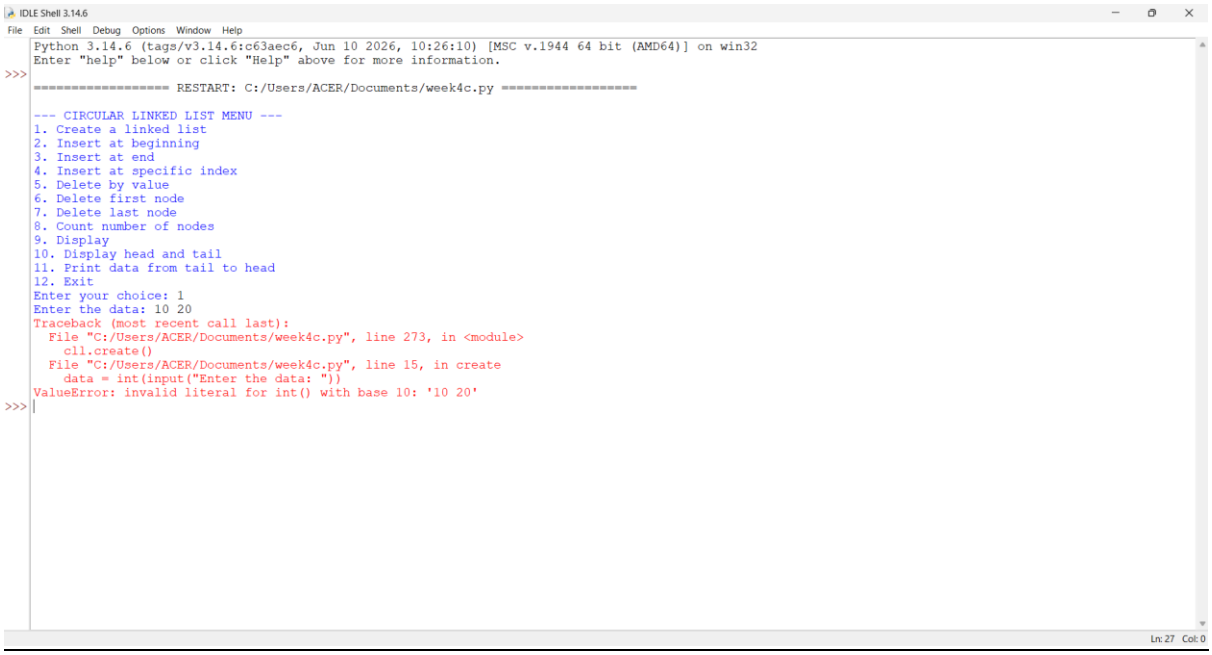
    elif choice == 10:
        cll.display_head_tail()

    elif choice == 11:
        cll.print_tail_to_head()

    elif choice == 12:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

Output:





EXPLANATION:

1. Node Class

- Each node stores:
    - data: the value.
    - next: pointer to the next node.
- In a circular linked list, the last node's next points back to the head, forming a circle.

2. Circular Linked List Class

This class manages the linked list. It starts with self.head = None (empty list). It provides several operations:

a. Create Linked List

- User specifies number of nodes.
- Each new node is linked at the end.
- The last node's next points back to the head, ensuring circularity.

b. Insert at Beginning

- A new node is created.
- It becomes the new head.
- The last node's next is updated to point to this new head.

c. Insert at End

- Traverse to the last node.
- Attach the new node after it.
- New node's next points back to head.

d. Insert at Specific Index

- If index = 0 → same as inserting at beginning.
- Otherwise, traverse to the given position.
- Insert the new node between two nodes.
- Maintain circularity by adjusting next pointers.

e. Delete by Value

- Search for the node with the given value.
- If it's the head:

- If only one node exists → set head to None.
  Otherwise, adjust last node's next to point to new head.

- If it's in the middle → bypass the node by linking previous node's next to the next node.

f. Delete First Node

- If only one node → set head to None.
- Otherwise, move head to the second node and update last node's next.

g. Delete Last Node

- Traverse to the second-last node.
- Make its next point to head.
- Removes the last node.

h. Count Nodes

- Traverse until you return to head.
- Count nodes during traversal.

i. Display

- Print nodes starting from head.
- Stop when traversal returns to head.
- Shows circularity by ending with "HEAD".

j. Display Head and Tail

- Head = first node.
- Tail = last node (whose next points to head).
- Prints both values.

k. Print Data from Tail to Head

- Find tail.
- Traverse backward by repeatedly searching for the node whose next points to the current node.
- Prints values in reverse order until reaching head.

3. Main Program (Menu-Driven)

- Runs an infinite loop with give options.
- Based on user choice, the corresponding method is executed.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Display on empty list.	Add condition: if self.head is None: → print "Circular Linked List is empty."
2.	Reverse traversal (Tail to Head) inefficient.	Current implementation repeatedly searches for predecessor. Rectify by using a doubly circular linked list if reverse traversal is frequent.

WEEK – 5

5a). AIM: Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

Program:

```
# Stack implementation using Array

stack = []

while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
```

```
print("3. Peek")
print("4. Display")
print("5. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter the data: "))
    stack.append(data)
    print("Element pushed successfully")

elif choice == 2:
    if len(stack) == 0:
        print("Stack Underflow")
    else:
        print("Popped element:", stack.pop())

elif choice == 3:
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print("Top element:", stack[-1])

elif choice == 4:
    if len(stack) == 0:
        print("Stack is empty")
    else:
```

```
            print("Stack elements:")
            for i in range(len(stack) - 1, -1, -1):
                print(stack[i])

    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

Output:



5b)Stack Using Linked List:

```
class Node:
    def __init__(self, data):
```

```
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self):
        data = int(input("Enter the data: "))

        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node

        print("Element pushed successfully")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped element:", self.top.data)
            self.top = self.top.next

    # Peek operation
```

```
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top

            print("Stack elements:")

            while temp is not None:
                print(temp.data)
                temp = temp.next



# Main program
s = Stack()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
```

```
print("3. Peek")
print("4. Display")
print("5. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    s.push()

elif choice == 2:
    s.pop()

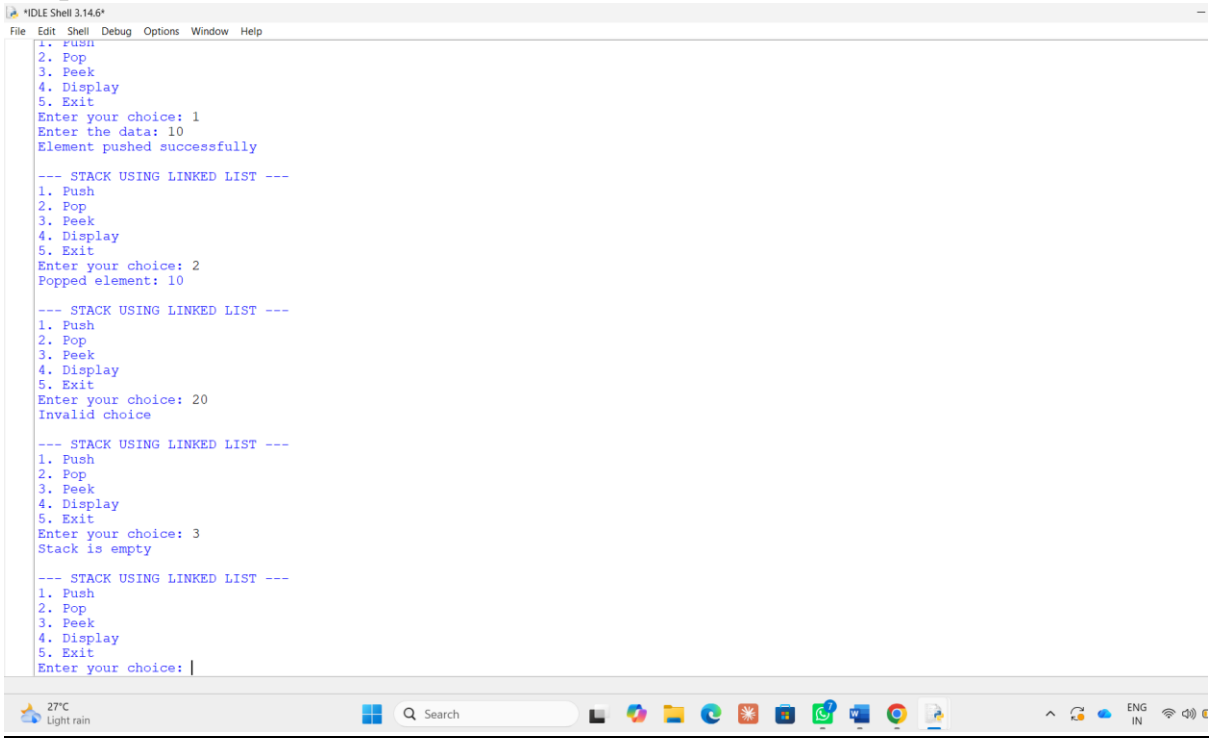
elif choice == 3:
    s.peek()

elif choice == 4:
    s.display()

elif choice == 5:
    print("Exiting program...")
    break

else:
    print("Invalid choice")
```

Output:



EXPLANATION:

Stack Using Array

Structure:

- Uses a Python list (self.stack) to store elements.
- self.top keeps track of the index of the last pushed element.
- self.size defines the maximum capacity.

Operations:

1. Push
    - Check if top == size - 1 → Overflow.
    - Otherwise, increment top and insert the element at that position.
2. Pop
    - Check if top == -1 → Underflow (empty stack).
    - Otherwise, remove the element at top, decrement top.
3. Peek
    - Shows the element at top without removing it.

4. Display
    - Prints elements from top down to 0 (top to bottom order).

Stack Using Linked List

Structure:

- Each element is a Node with:
    - data (value)
    - next (pointer to the next node).
- self.top points to the most recently pushed node.

Operations:

1. Push
    - Create a new node.
    - Link its next to the current top.
    - Update top to this new node.
2. Pop
    - If top is None → Underflow.
    - Otherwise, remove the node at top and update top to top.next.
3. Peek
    - Shows the data at top.
4. Display
    - Traverse from top downwards, printing each node's data.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	def _init_(self, size):	Python expects **two underscores** before and after init.
2.	Display on empty stack.	Add condition: if self.top is None: → print "Stack is empty".

3.	Syntax error	Later it was rectified in the for loop of display section.

Week-6

6a)Aim: To write a Python program to implement a queue using an array (list) and linked list and perform insertion (Enqueue), deletion (Dequeue), Peek, and Display operations using the FIFO (First In, First Out) principle.

Program:(Queue using array).

```
queue=[]
size=int(input("Enter the size of the queue:"))

while True:
    print(" \nQueue Menu:")
    print("1.Enqueue")
    print("2.Dequeue")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")

    choice=int(input("Enter your choice form 1-5:"))

    if choice==1:
        if len(queue)==size:
            print("Queue is full.Cant add elements into the queue")
        else:
            value=int(input("Enter the value to be added into the queue:"))
```

```
        queue.append(value)
        print("Value inserted")

elif choice==2:
    if len(queue)==0:
        print("There are no elements to delete in the queue")
    else:
        value=queue.pop(0)
        print("Value deleted:",value)

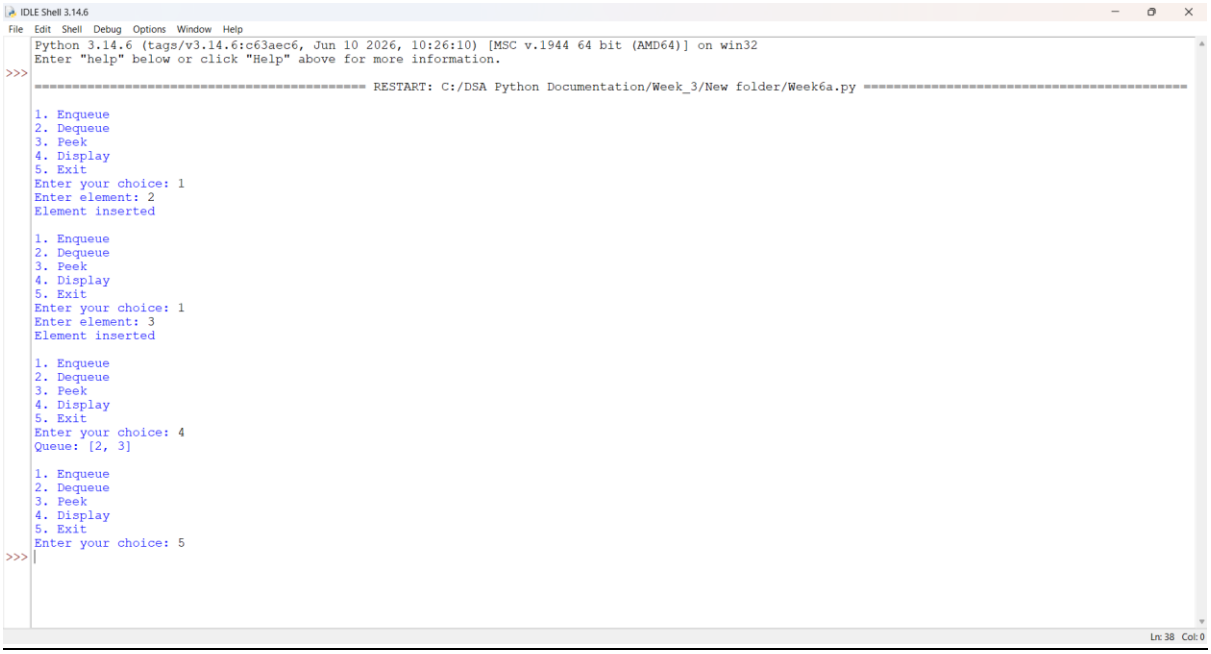
elif choice==3:
    if len(queue)==0:
        print("There are no elements no print in the queue")
    else:
        print("First element is:",queue[0])

elif choice==4:
    if len(queue)==0:
        print("There are no elements to display")
    else:
        print("Queue:",queue)

elif choice==5:
    print("Exiting...")

else:
    print("invalid choice")
```

Output:



Explanation:

1.Queue Initialization: An empty list queue = [] is created to store elements, and size = 5 specifies the maximum queue capacity.

2. Menu Display: A while True loop displays the menu options: Enqueue, Dequeue, Peek, Display, and Exit.

3. Enqueue (Insertion): If the queue is not full, the program accepts an element from the user and inserts it at the rear using append().

4. Dequeue (Deletion): If the queue is not empty, the program removes and displays the front element using pop(0).

5. Peek Operation: The program displays the front element using queue[0] without removing it.

6. Display Operation: The program prints all the elements present in the queue. If the queue is empty, it displays a message.

7. Exit Operation: When the user selects option 5, the break statement terminates the loop and ends the program.

8. Invalid Choice: If the user enters an option other than 1 to 5, the program displays "Invalid choice".

Error Tables:

S.No	Errors obtained	Rectification
01.	Queue Overflow	Occurs when inserting an element into a full queue. Check whether len(queue) == size before insertion.
02.	Index Error	Occurs when accessing queue[0] while the queue is empty. Check whether the queue is empty before accessing the front element.

Program(Queue using linked list):

```
# Node class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


# Queue class
class Queue:
    def __init__(self):
        self.front = None
```

```
    self.rear = None

# Add element to queue
def enqueue(self, data):
    new_node = Node(data)

    if self.rear is None:
        self.front = self.rear = new_node
    else:
        self.rear.next = new_node
        self.rear = new_node

# Remove element from queue
def dequeue(self):
    if self.front is None:
        print("Queue is empty")
        return None

    data = self.front.data
    self.front = self.front.next

    if self.front is None:
        self.rear = None

    return data

# Display queue
```

```
    def display(self):
        if self.front is None:
            print("Queue is empty")
            return

        current = self.front
        while current:
            print(current.data, end=" ")
            current = current.next
        print()

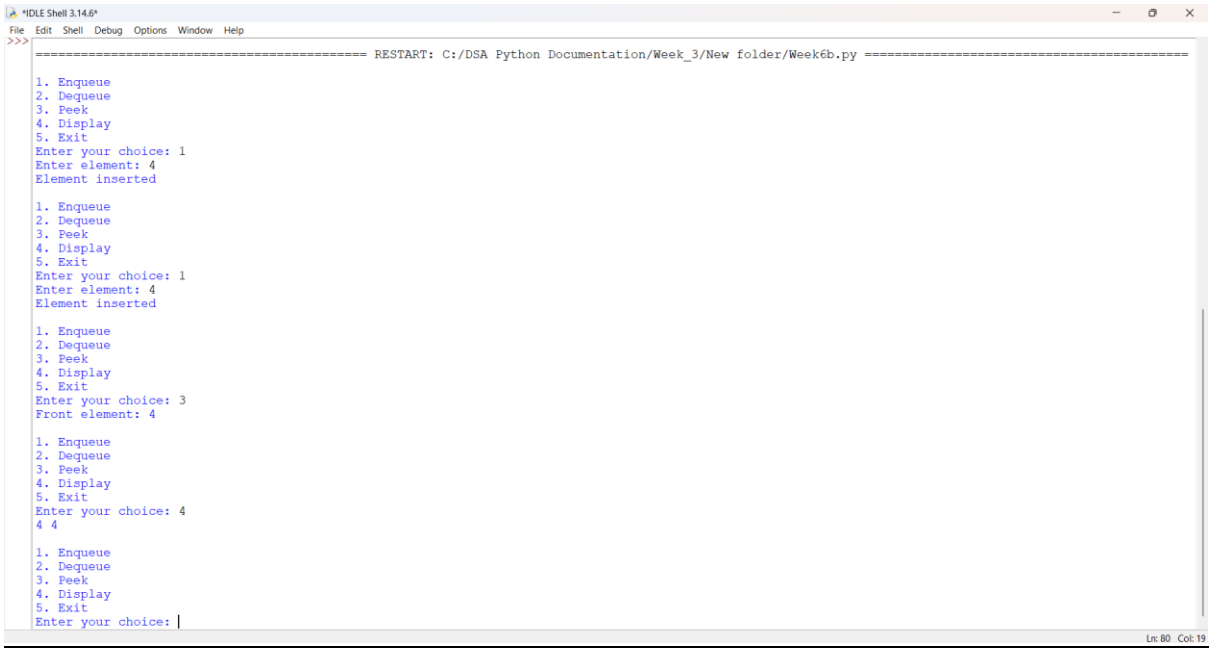

# Example
q = Queue()

q.enqueue(10)
q.enqueue(20)
q.enqueue(30)

print("Queue:")
q.display()

print("Deleted:", q.dequeue())
print("Queue after dequeue:")
q.display()
```

Output:



Explanation:

1. Node Creation: The Node class stores the element in data and the address of the next node in next.

2. Initialization: The variables front and rear are initialized to None, indicating that the queue is empty.

3. Menu Display: A while True loop displays the menu options: Enqueue, Dequeue, Peek, Display, and Exit.

4. Enqueue (Insertion): A new node is created with the given element. If the queue is empty, both front and rear point to the new node. Otherwise, the new node is linked at the rear, and rear is updated.

5. Dequeue (Deletion): If the queue is empty, the program displays "Queue Underflow". Otherwise, the front element is deleted by updating front = front.next. If the queue becomes empty, rear is also set to None.

6. Peek Operation: The program displays the front element using front.data without removing it.

7. Display Operation: A temporary variable temp starts at front and traverses each node using temp = temp.next until it reaches None, printing all elements.

8. Exit Operation: When the user selects option 5, the break statement terminates the loop.

9. Invalid Choice: If the user enters an option other than 1 to 5, the program displays "Invalid choice".

Error Table:

S.No	Error Obtained	Rectification
01	No Errors	----------

6b)Aim:To implement circular queue using array

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None


    def enqueue(self, data):
        new_node = Node(data)


        if self.front is None:
            self.front = self.rear = new_node
            self.rear.next = self.front
```

```
        else:
            self.rear.next = new_node
            self.rear = new_node
            self.rear.next = self.front

        print("Element inserted:", data)

    def dequeue(self):
        if self.front is None:
            print("Queue is Empty")
        elif self.front == self.rear:
            print("Deleted Element:", self.front.data)
            self.front = self.rear = None
        else:
            print("Deleted Element:", self.front.data)
            self.front = self.front.next
            self.rear.next = self.front

    def peek(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print("Front Element:", self.front.data)
```

```
    def display(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            temp = self.front
            print("Circular Queue Elements:")
            while True:
                print(temp.data, end=" ")
                temp = temp.next
                if temp == self.front:
                    break
            print()


cq = CircularQueue()

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

```
    ch = int(input("Enter your choice: "))

    if ch == 1:
        data = int(input("Enter Element: "))
        cq.enqueue(data)
    elif ch == 2:
        cq.dequeue()
    elif ch == 3:
        cq.peek()
    elif ch == 4:
        cq.display()
    elif ch == 5:
        print("Exiting...")
        break
    else:
        print("Invalid Choice")
```

Output:



Explanation

1. Node Class: Creates a node with two parts: data and a pointer to the next node.

2. Constructor (__init__): Initializes the front and rear pointers as None, indicating that the queue is empty.

3. Enqueue Operation: Creates a new node and inserts it at the rear end of the queue. The rear node points back to the front node to maintain the circular structure.

4. Dequeue Operation: Deletes the element from the front of the queue. If only one node is present, both front and rear are set to None.

5. Peek Operation: Displays the front element without removing it from the queue.

6. Display Operation: Traverses the linked list from front to rear and displays all elements until it reaches the front node again.

7. Main Program: Displays a menu with Enqueue, Dequeue, Peek, Display, and Exit options. The user selects an operation to perform.

Error Table:

S.No	Errors obtained	Rectification
01	Attribute Error	Maintain proper indentation in Python code
02	Recursion Error	Check the node attributes and function names
03	Type Error	Enter valid integer values for tree operations.

Week-7

7a) Aim: Python program to implement binary tree using array, linked list with all appix operations (Insertion,deletion,traversal,display,searching).

Program:(Using Array):

```
tree = [None] * 100

n = 0


def insert(x):
    global n
    if n == 100:
        print("Tree is full")
    else:
        tree[n] = x
        n += 1
        print("Inserted successfully")


def delete(x):
    global n
```

```
    if n == 0:
        print("Tree is empty")
        return

    if x not in tree[:n]:
        print("Element not found")
        return

    i = tree.index(x)
    tree[i] = tree[n - 1]
    tree[n - 1] = None
    n -= 1
    print("Deleted successfully")

def search(x):
    if x in tree[:n]:
        print("Element found at index", tree.index(x))
    else:
        print("Element not found")

def inorder(i):
    if i < n:
        inorder(2 * i + 1)
```

```
        print(tree[i], end=" ")
        inorder(2 * i + 2)

def preorder(i):
    if i < n:
        print(tree[i], end=" ")
        preorder(2 * i + 1)
        preorder(2 * i + 2)

def postorder(i):
    if i < n:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
        print(tree[i], end=" ")

def display():
    if n == 0:
        print("Tree is empty")
    else:
        for i in range(n):
            print(i, ":", tree[i])

while True:
```

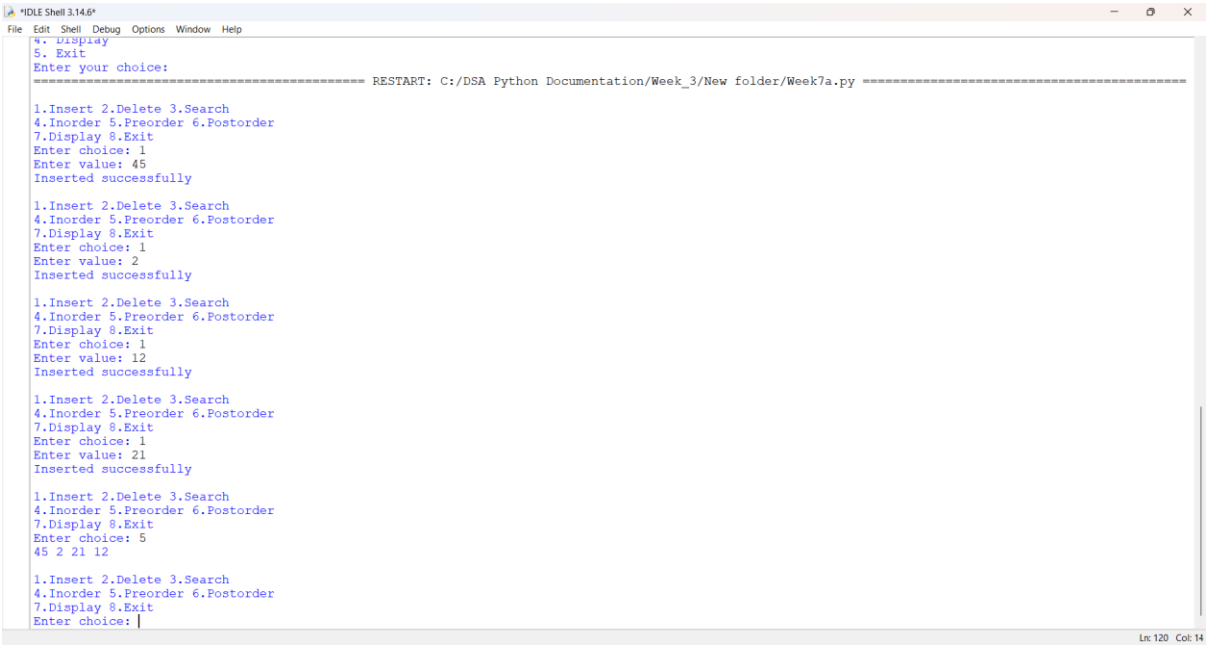
```
print("\n1.Insert 2.Delete 3.Search")
print("4.Inorder 5.Preorder 6.Postorder")
print("7.Display 8.Exit")

ch = int(input("Enter choice: "))

if ch == 1:
    x = int(input("Enter value: "))
    insert(x)
elif ch == 2:
    x = int(input("Enter value to delete: "))
    delete(x)
elif ch == 3:
    x = int(input("Enter value to search: "))
    search(x)
elif ch == 4:
    inorder(0)
    print()
elif ch == 5:
    preorder(0)
    print()
elif ch == 6:
    postorder(0)
```

```
        print()
    elif ch == 7:
        display()
    elif ch == 8:
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

1. Array creation: tree = [None] * 100 creates an array with 100 positions to store the tree elements. The variable n keeps track of the number of elements.

2. Insertion: The insert(x) function inserts the given element at the next available array position and increases n by one.

3. Deletion: The delete(x) function searches for the given element, replaces it with the last element in the array and removes the last element.

4. Searching: The search(x) function checks whether the given element exists in the array. If found, it displays the index; otherwise, it displays "Element not found".

5. Inorder traversal: The inorder(i) function visits the left subtree, root and right subtree in that order.

6. Preorder traversal: The preorder(i) function visits the root, left subtree and right subtree in that order.

7. Postorder traversal: The postorder(i) function visits the left subtree, right subtree and root in that order.

8. Display: The display() function prints all stored elements along with their array indices.

9. Menu-driven operations: The while True loop displays a menu and allows the user to select an operation. The program continues until the user selects Exit.

Array index formulas

For a node at index i:

- Left child: 2*i + 1
- Right child: 2*i + 2
- Parent: (i-1)//2

Error Table:

S.No	Errors Obtained	Rectification
01	IndentationError: expected an indented block	Correct the indentation of statements inside functions and loops.
02	IndexError: list index out of range	Check array boundaries before accessing an index.

7b)Program (using linked list):

```
class Node:

    def __init__(self, data):

        self.data = data
```

```
        self.left = None
        self.right = None

root = None

def insert(x):
    global root
    new = Node(x)

    if root is None:
        root = new
        print("Inserted successfully")
        return

    q = [root]
    while q:
        temp = q.pop(0)

        if temp.left is None:
            temp.left = new
            break
        else:
            q.append(temp.left)
```

```
            if temp.right is None:
                temp.right = new
                break
            else:
                q.append(temp.right)

        print("Inserted successfully")

    def search(x):
        if root is None:
            print("Tree is empty")
            return

        q = [root]
        while q:
            temp = q.pop(0)

            if temp.data == x:
                print("Element found")
                return

            if temp.left:
```

```
            q.append(temp.left)
        if temp.right:
            q.append(temp.right)

    print("Element not found")

def delete(x):
    global root
    if root is None:
        print("Tree is empty")
        return

    q = [(root, None)]
    target = None
    last = None
    parent = None

    while q:
        temp, par = q.pop(0)

        if temp.data == x and target is None:
            target = temp
```

```
    last = temp
    parent = par

    if temp.left:
        q.append((temp.left, temp))
    if temp.right:
        q.append((temp.right, temp))

if target is None:
    print("Element not found")
    return

target.data = last.data

if parent is None:
    root = None
elif parent.right == last:
    parent.right = None
else:
    parent.left = None

print("Deleted successfully")
```

```
def inorder(node):
    if node:
        inorder(node.left)
        print(node.data, end=" ")
        inorder(node.right)

def preorder(node):
    if node:
        print(node.data, end=" ")
        preorder(node.left)
        preorder(node.right)

def postorder(node):
    if node:
        postorder(node.left)
        postorder(node.right)
        print(node.data, end=" ")

def display():
    if root is None:
        print("Tree is empty")
        return
```

```
    q = [root]
    while q:
        temp = q.pop(0)
        print(temp.data, end=" ")

        if temp.left:
            q.append(temp.left)
        if temp.right:
            q.append(temp.right)
    print()

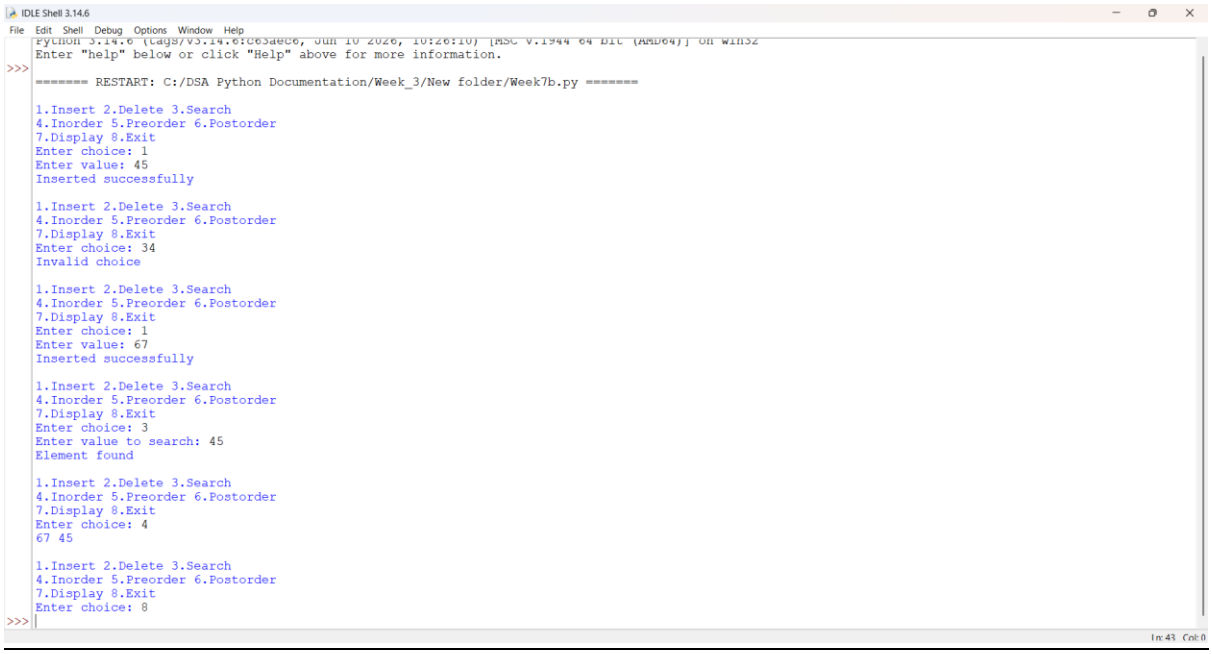
while True:
    print("\n1.Insert 2.Delete 3.Search")
    print("4.Inorder 5.Preorder 6.Postorder")
    print("7.Display 8.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        insert(x)
    elif ch == 2:
        x = int(input("Enter value to delete: "))
```

```
        delete(x)
    elif ch == 3:
        x = int(input("Enter value to search: "))
        search(x)
    elif ch == 4:
        inorder(root)
        print()
    elif ch == 5:
        preorder(root)
        print()
    elif ch == 6:
        postorder(root)
        print()
    elif ch == 7:
        display()
    elif ch == 8:
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

1. Node creation: The Node class creates a node containing data, left and right. Initially, both child references are None.

2. Root initialization: The variable root stores the address (reference) of the root node. Initially, the tree is empty.

3. Insertion: The insert(x) function creates a new node. If the tree is empty, the new node becomes the root. Otherwise, it uses a queue to find the first available position and inserts the node there.

4. Searching: The search(x) function visits nodes level by level using a queue. It compares each node's data with the required value and displays whether the element is found.

5. Deletion: The delete(x) function finds the node to delete and the deepest, rightmost node. It copies the deepest node's data into the target node and removes the deepest node.

6. Inorder traversal: The inorder(node) function visits the left subtree, root and right subtree.

7. Preorder traversal: The preorder(node) function visits the root, left subtree and right subtree.

8. Postorder traversal: The postorder(node) function visits the left subtree, right subtree and root.

9. Display: The display() function uses a queue to print all nodes level by level.

10. Menu-driven operations: The while True loop allows the user to select insertion, deletion, searching, traversal, display or exit.

- Inorder: 2 1 3
- Preorder: 1 2 3
- Postorder: 2 3 1
- Level-order display: 1 2 3

Error Table:

S.No	Errors Obtained	Rectification
01	NameError: name 'root' is not defined	Initialize root = None before using it.
02	ValueError: invalid literal for int()	Enter a valid integer when prompted for input.

Week-8

8a)AIM: Write a python program to implement Binary Search tree with its operation.

Program:

```
class Node:

    def __init__(self, data):

        self.data = data

        self.left = None

        self.right = None
```

```
class BST:
    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)
        else:
            root.right = self.insert(root.right, data)

        return root

    def search(self, root, key):
        if root is None or root.data == key:
            return root

        if key < root.data:
            return self.search(root.left, key)

        return self.search(root.right, key)

    def inorder(self, root):
        if root:
```

```
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)

def preorder(self, root):
    if root:
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

def postorder(self, root):
    if root:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")

def find_min(self, root):
    while root.left:
        root = root.left
    return root

def delete(self, root, key):
    if root is None:
```

```
            return root

        if key < root.data:
            root.left = self.delete(root.left, key)

        elif key > root.data:
            root.right = self.delete(root.right, key)

        else:
            if root.left is None:
                return root.right

            elif root.right is None:
                return root.left

            temp = self.find_min(root.right)
            root.data = temp.data
            root.right = self.delete(root.right, temp.data)

        return root


bst = BST()
```

```
root = None

while True:
    print("\n\n--- BINARY SEARCH TREE ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        data = int(input("Enter value: "))
        root = bst.insert(root, data)
        print("Node inserted successfully")

    elif ch == 2:
        key = int(input("Enter value to search: "))
        if bst.search(root, key):
            print("Element found")
```

```
        else:
            print("Element not found")

    elif ch == 3:
        key = int(input("Enter value to delete: "))
        root = bst.delete(root, key)
        print("Node deleted successfully")

    elif ch == 4:
        print("Inorder Traversal:")
        bst.inorder(root)

    elif ch == 5:
        print("Preorder Traversal:")
        bst.preorder(root)

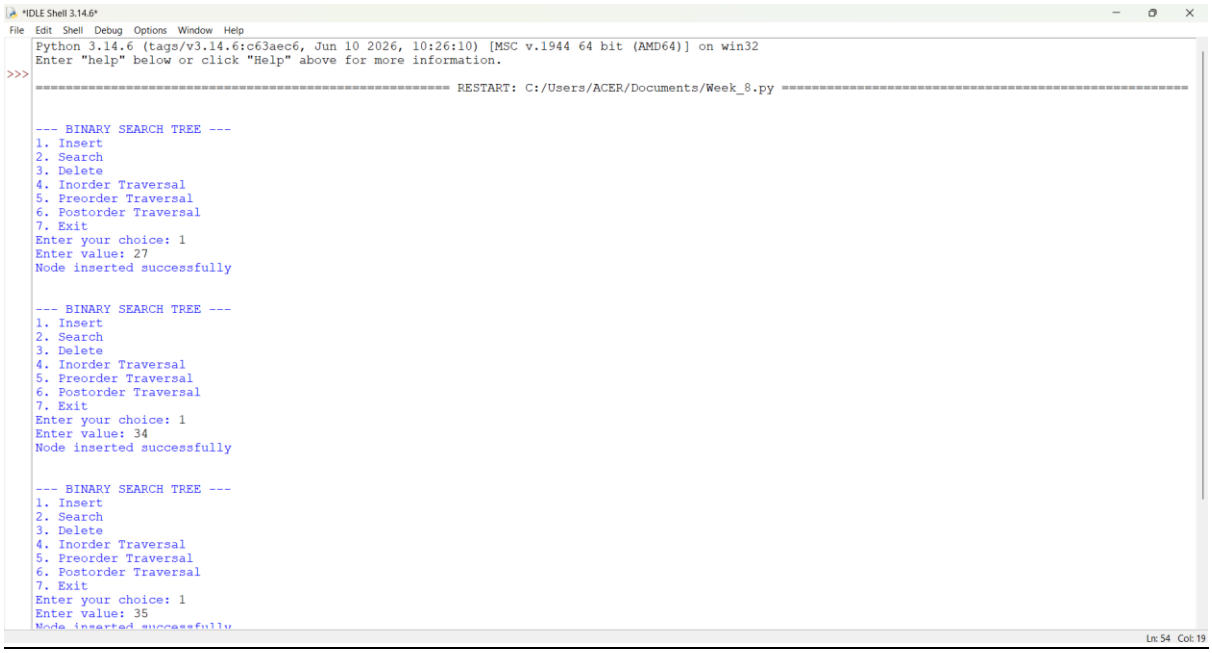
    elif ch == 6:
        print("Postorder Traversal:")
        bst.postorder(root)

    elif ch == 7:
        print("Exiting...")
        break
```

```
    else:
        print("Invalid choice")
```

Output:

PositiveCase:







NegativeCase:



Explanation:

1. Node Creation: The Node class is used to create a new node containing data, left pointer and right pointer.

2. BST Class: The BST class contains all the operations performed on the Binary Search Tree.

3. Insertion: The insert() function inserts a new node into the tree. If the value is smaller than the root, it is inserted on the left side; otherwise, it is inserted on the right side.

4. Searching: The search() function searches for a given element. It compares the key with the root and moves either left or right until the element is found.

5. Deletion: The delete() function removes a node from the tree. It handles three cases: a node with no children, one child, or two children.

6. Find Minimum: The find_min() function finds the smallest value in the right subtree. It is used during deletion when a node has two children.

7. Inorder Traversal: The inorder() function visits the left subtree, root node and right subtree. It displays the elements in ascending order.

8. Preorder Traversal: The preorder() function visits the root node, left subtree and right subtree.

9. Postorder Traversal: The postorder() function visits the left subtree, right subtree and root node.

10. Menu-Driven Operations: A while loop displays the menu and allows the user to select insertion, searching, deletion, traversal or exit.

11. Exit: When the user selects option 7, the break statement terminates the program.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Recursion Depth Error (Recursion Error):** Deep or skewed trees exceed Python's default recursion limit (usually 1000 calls) during recursive traversals (inorder, preorder, postorder).	Rewrite traversal functions iteratively using an explicit stack, or increase the recursion limit using sys.setrecursionlimit().

8b)Aim:To implement AVL tree and perform the basic operations

Program:

```
 class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

class AVLTree:
    def height(self, root):
        if root is None:
            return 0
        return root.height

    def balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    def rightRotate(self, y):
        x = y.left
        T2 = x.right

        x.right = y
```

```
        y.left = T2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))

        return x

    def leftRotate(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))

        return y
```

```
def insert(self, root, key):
    if root is None:
        return Node(key)

    if key < root.key:
        root.left = self.insert(root.left, key)
    elif key > root.key:
        root.right = self.insert(root.right, key)
    else:
        return root

    root.height = 1 + max(self.height(root.left),
                          self.height(root.right))

    balance = self.balance(root)

    if balance > 1 and key < root.left.key:
        return self.rightRotate(root)

    if balance < -1 and key > root.right.key:
        return self.leftRotate(root)

    if balance > 1 and key > root.left.key:
```

```
            root.left = self.leftRotate(root.left)
            return self.rightRotate(root)

        if balance < -1 and key < root.right.key:
            root.right = self.rightRotate(root.right)
            return self.leftRotate(root)

        return root

    def minValueNode(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current

    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
```

```
else:
    if root.left is None:
        return root.right
    elif root.right is None:
        return root.left

    temp = self.minValueNode(root.right)
    root.key = temp.key
    root.right = self.delete(root.right, temp.key)

if root is None:
    return root

root.height = 1 + max(self.height(root.left),
                      self.height(root.right))

balance = self.balance(root)

if balance > 1 and self.balance(root.left) >= 0:
    return self.rightRotate(root)

if balance > 1 and self.balance(root.left) < 0:
    root.left = self.leftRotate(root.left)
```

```
            return self.rightRotate(root)

        if balance < -1 and self.balance(root.right) <= 0:
            return self.leftRotate(root)

        if balance < -1 and self.balance(root.right) > 0:
            root.right = self.rightRotate(root.right)
            return self.leftRotate(root)

        return root

    def search(self, root, key):
        if root is None or root.key == key:
            return root

        if key < root.key:
            return self.search(root.left, key)

        return self.search(root.right, key)

    def inorder(self, root):
        if root:
            self.inorder(root.left)
```

```
            print(root.key, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")


tree = AVLTree()
root = None

while True:
    print("\n\n1. Insert")
    print("2. Delete")
    print("3. Search")
```

```
print("4. Inorder Traversal")
print("5. Preorder Traversal")
print("6. Postorder Traversal")
print("7. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    key = int(input("Enter value to insert: "))
    root = tree.insert(root, key)

elif choice == 2:
    key = int(input("Enter value to delete: "))
    root = tree.delete(root, key)

elif choice == 3:
    key = int(input("Enter value to search: "))
    if tree.search(root, key):
        print("Element found")
    else:
        print("Element not found")

elif choice == 4:
```

```
        print("Inorder Traversal:")
        tree.inorder(root)

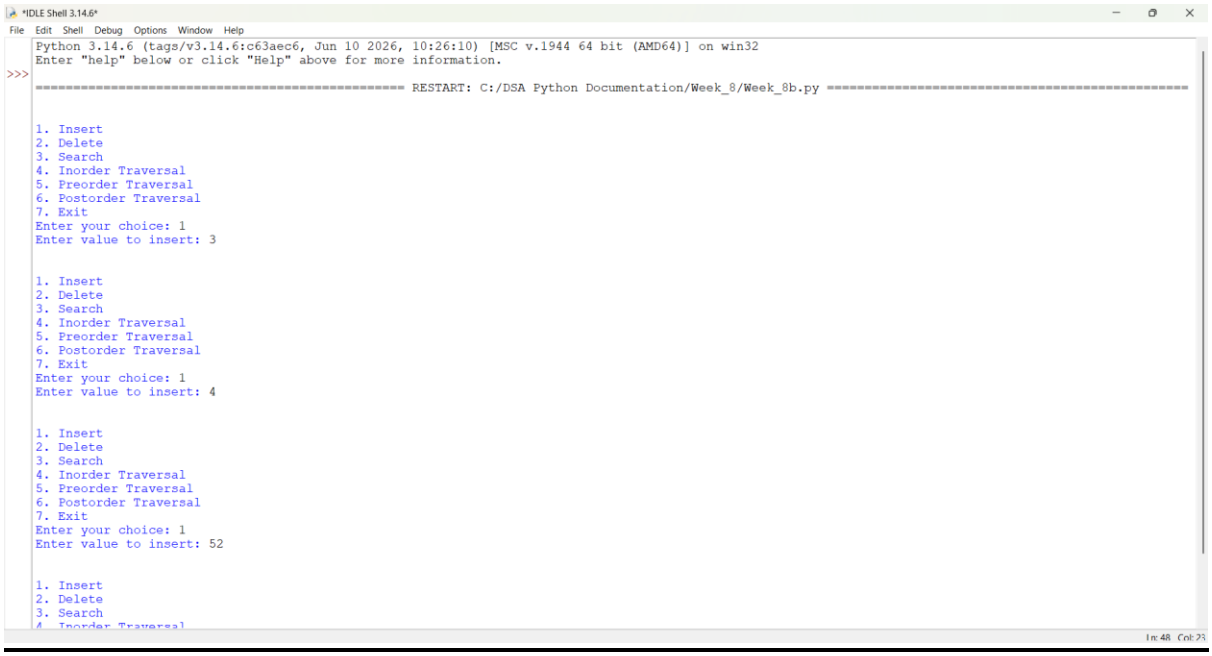
    elif choice == 5:
        print("Preorder Traversal:")
        tree.preorder(root)

    elif choice == 6:
        print("Postorder Traversal:")
        tree.postorder(root)

    elif choice == 7:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

Output:





Explanation:

1. Node Creation: A Node class is created to store the key value, left child, right child and height of each node.

2. AVL Tree Class: The AVLTree class contains all the functions required to perform AVL tree operations.

3. Height Calculation: The height() function calculates the height of a node. If the node is empty, it returns 0.

4. Balance Factor: The balance() function calculates the difference between the heights of the left and right subtrees. The balance factor must be -1, 0 or +1.

5. Right Rotation: The rightRotate() function performs right rotation to balance the tree when the left subtree becomes heavy.

6. Left Rotation: The leftRotate() function performs left rotation to balance the tree when the right subtree becomes heavy.

7. Insertion: The insert() function adds a new element following Binary Search Tree rules and checks the balance factor to perform the required rotations.

8. Minimum Value Node: The minValueNode() function finds the smallest value in the right subtree. It is mainly used during deletion.

9. Deletion: The delete() function removes the specified element and updates the height and balance factor of the remaining nodes.

10. Searching: The search() function searches for an element by comparing it with the root node and moving towards the left or right subtree.

11. Inorder Traversal: The inorder() function visits the left subtree, root node and right subtree. It displays elements in sorted order.

12. Preorder Traversal: The preorder() function visits the root node, left subtree and right subtree.

13. Postorder Traversal: The postorder() function visits the left subtree, right subtree and root node.

14. Menu-Driven Operations: The main program displays a menu to perform insertion, deletion, searching and traversal operations based on the user's choice.

15. Exit: The program terminates when the user selects option 7.

Error Table:

S.No	Errors obtained	Rectification
01	Attribute Error	Maintain proper indentation in Python code
02	Recursion Error	Check the node attributes and function names
03	Type Error	Enter valid integer values for tree operations.

Week-9

Practice Question:(Week-9a):

Aim: To implement hashing using separate chaining , linear and quadratic probing.

Program:(Using array):

```
# Hashing using Separate Chaining, Linear Probing and Quadratic Probing


size = 10


# Separate Chaining

chain = [[] for i in range(size)]


def separate_chaining(key):

    index = key % size

    chain[index].append(key)


# Linear Probing

linear = [None] * size


def linear_probing(key):

    index = key % size


    while linear[index] is not None:
```

```
        index = (index + 1) % size

    linear[index] = key

# Quadratic Probing
quadratic = [None] * size

def quadratic_probing(key):
    index = key % size
    i = 0

    while quadratic[(index + i * i) % size] is not None:
        i += 1

        if i == size:
            print("Hash table is full")
            return

    quadratic[(index + i * i) % size] = key



while True:
    print("\n1. Separate Chaining")
```

```
print("2. Linear Probing")
print("3. Quadratic Probing")
print("4. Display All")
print("5. Exit")

ch = int(input("Enter your choice: "))

if ch == 1:
    key = int(input("Enter key: "))
    separate_chaining(key)
    print("Key inserted successfully")

elif ch == 2:
    key = int(input("Enter key: "))
    linear_probing(key)
    print("Key inserted successfully")

elif ch == 3:
    key = int(input("Enter key: "))
    quadratic_probing(key)

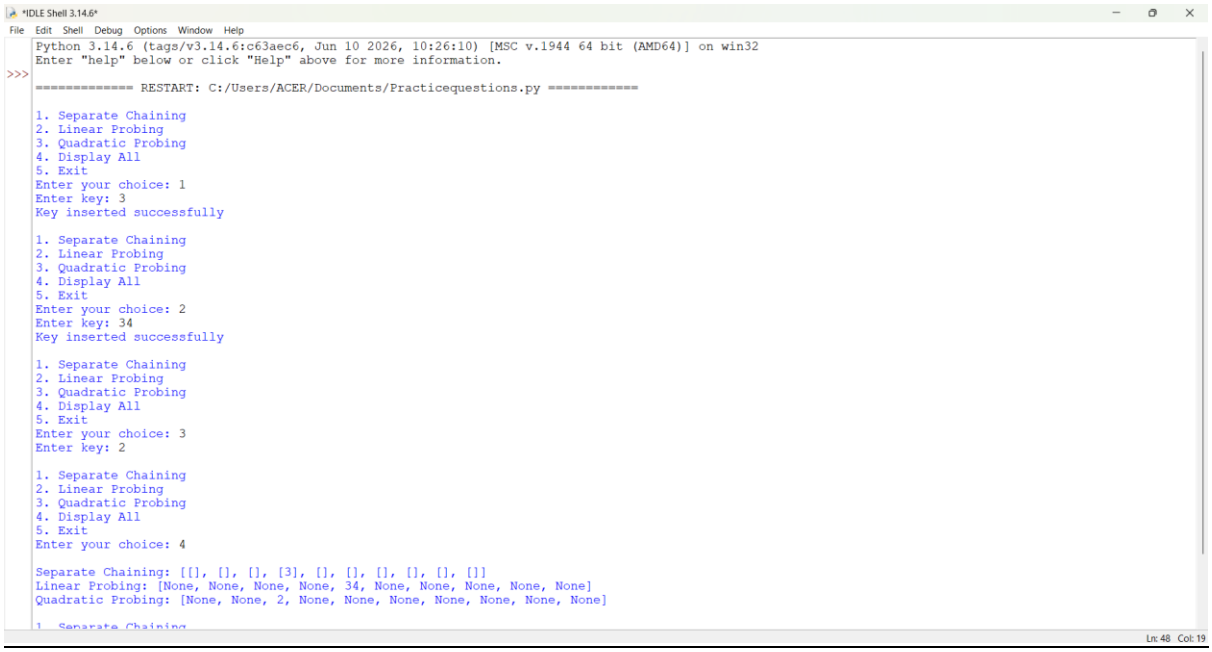
elif ch == 4:
    print("\nSeparate Chaining:", chain)
```

```
        print("Linear Probing:", linear)
        print("Quadratic Probing:", quadratic)

    elif ch == 5:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

Output:



Explanation:

1. Hash Table Creation: A hash table of size 10 is created to store the keys. Separate lists are used for Separate Chaining, Linear Probing and Quadratic Probing.

2. Hash Function: The hash function key % size calculates the index where the key should be stored in the hash table.

3. Separate Chaining: The separate_chaining() function handles collisions by storing multiple keys in a list at the same index.

4. Linear Probing: The linear_probing() function checks the next available position whenever a collision occurs. It continues checking until an empty position is found.

5. Quadratic Probing: The quadratic_probing() function handles collisions by checking positions using the square of the probing number (i * i) until an empty position is found.

6. Menu Display: A while loop displays the menu and allows the user to select the required hashing technique.

7. Insertion: The user enters a key, and the selected hashing technique inserts it into the corresponding hash table.

8. Display: The display option prints all three hash tables so that the user can see how the keys are stored.

9. Exit: When the user selects option 5, the break statement terminates the program.

Important Formulae:-

Method	**Formula**
Division	$h(x) = x \% M$
Mid-Square	Square the key and extract middle digits
Digit Folding	Add the separate parts of the key
Multiplication	$h(x) = \text{floor}(M \times \text{fractional}(x \times C))$
Linear Probing	$(h(x) + i) \% M$
Quadratic Probing	$(h(x) + i^2) \% M$
Double Hashing	$(h1(x) + i \times h2(x)) \% M$

Method	Formula
Load Factor	$\alpha = n / M$

Time complexity:-

Operation	Average Case	Worst Case
Insertion	O(1)	O(n)
Searching	O(1)	O(n)
Deletion	O(1)	O(n)
Display	O(n + M)	O(n + M)
Load Factor	O(n + M)	O(n + M)
Rehashing	O(n)	O(n)

SPACE COMPLEXITY: O(n + M)

Error table:

S.No	Errors Obtained	Rectification
1	Index out of range	Used modulo (%) to calculate valid indexes.
2	Collision occurred during insertion	Handled collisions using Separate Chaining.
3	Linear probing failed to find an empty position.	Checked the next available index.
4	Quadratic probing collision	Used the quadratic probing formula to find an empty position.

9b)Using linked list:

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    def insert(self, key):
        index = key % self.size
        new_node = Node(key)

        if self.table[index] is None:
            self.table[index] = new_node
        else:
            temp = self.table[index]

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
```

```
        print("Key inserted successfully")

    def search(self, key):
        index = key % self.size
        temp = self.table[index]

        while temp is not None:
            if temp.data == key:
                return True
            temp = temp.next

        return False

    def display(self):
        for i in range(self.size):
            print(i, end=": ")
            temp = self.table[i]

            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next

            print("None")


ht = HashTable(10)
```

```
while True:
    print("\n--- HASHING USING LINKED LIST ---")
    print("1. Insert")
    print("2. Search")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        key = int(input("Enter key: "))
        ht.insert(key)

    elif ch == 2:
        key = int(input("Enter key to search: "))

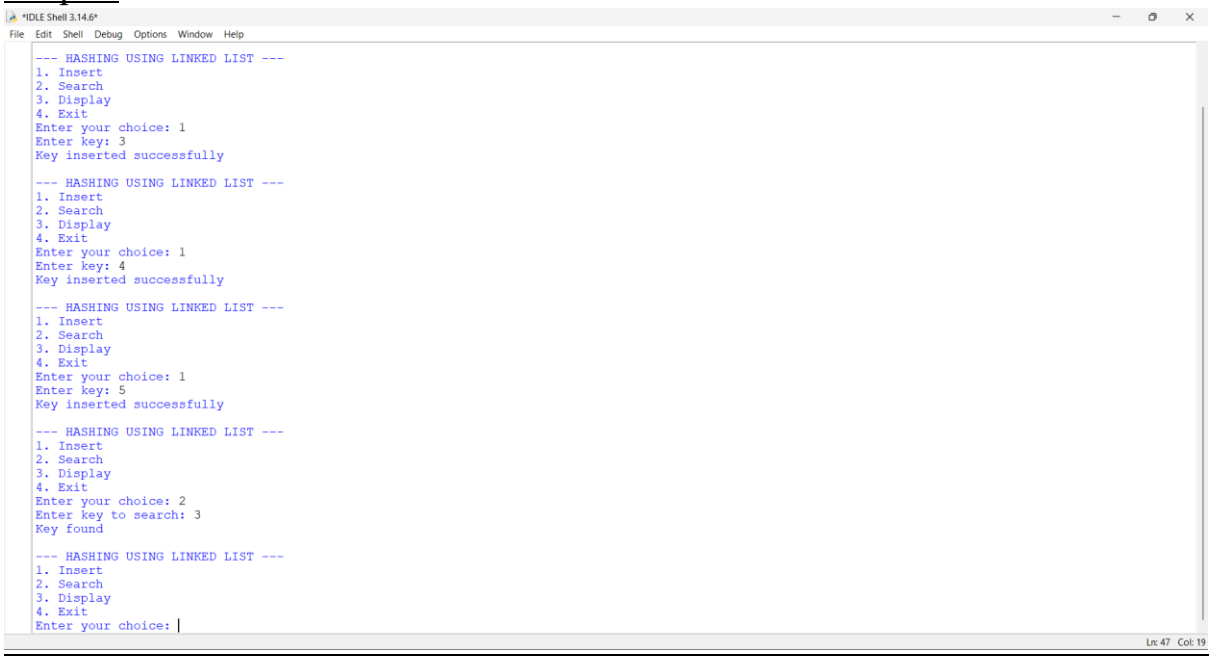
        if ht.search(key):
            print("Key found")
        else:
            print("Key not found")

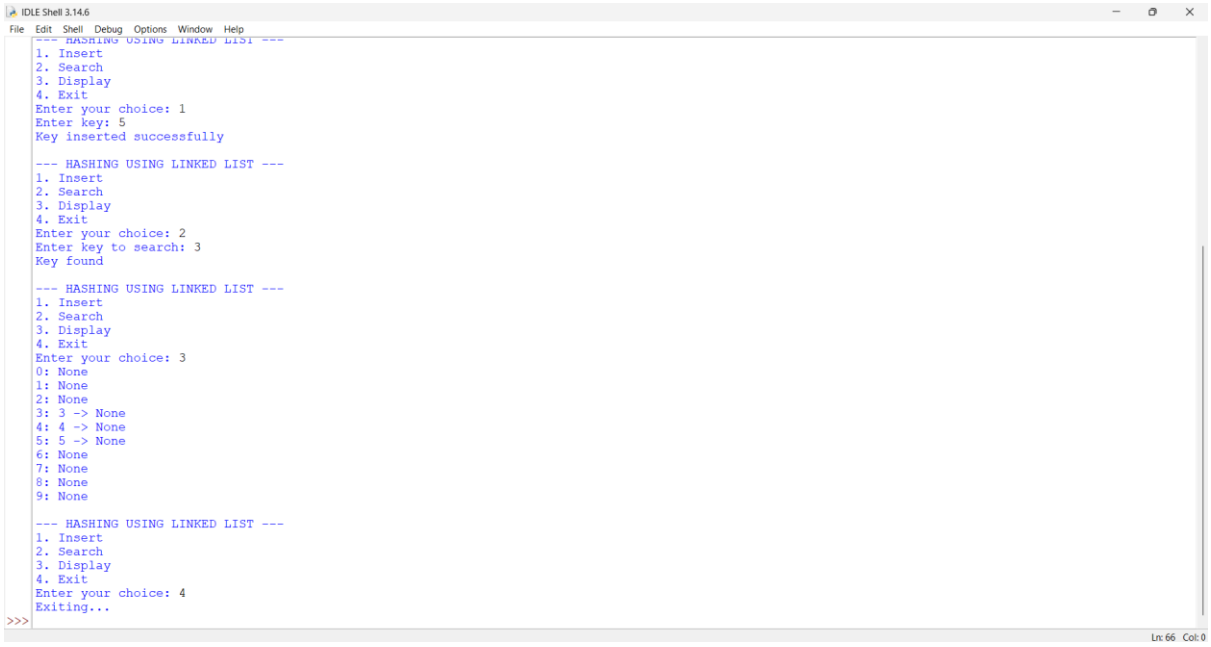
    elif ch == 3:
        ht.display()

    elif ch == 4:
        print("Exiting...")
        break
```

```
    else:
        print("Invalid choice")
```

Output:





Explanation:

1. Node Creation: The Node class is used to create a node containing data and a next pointer that stores the address of the next node.

2. Hash Table Creation: The HashTable class creates a hash table of size 10 using an array of linked-list pointers.

3. Initialization: The __init__() function initializes the hash table with all positions set to None.

4. Insertion: The insert() function calculates the index using the hash function key % size and inserts the key into the corresponding linked list.

5. Collision Handling: If two or more keys get the same index, they are stored in the same linked list. This method is called Separate Chaining.

6. Searching: The search() function searches for a given key by traversing the linked list at the calculated index.

7. Display: The display() function displays all the hash table indexes and their corresponding linked-list elements.

8. Menu-Driven Operations: A while loop displays the menu and allows the user to select insertion, searching, display or exit.

9. Exit: When the user selects option 4, the break statement terminates the program.

Important Formulae:-

Method	**Formula**
Division	$h(x) = x \% M$
Mid-Square	Square the key and extract middle digits
Digit Folding	Add the separate parts of the key
Multiplication	$h(x) = \text{floor}(M \times \text{fractional}(x \times C))$
Linear Probing	$(h(x) + i) \% M$
Quadratic Probing	$(h(x) + i^2) \% M$
Double Hashing	$(h1(x) + i \times h2(x)) \% M$
Load Factor	$\alpha = n / M$

Time complexity:-

Operation	Average Case	Worst Case
Insertion	O(1)	O(n)
Searching	O(1)	O(n)
Deletion	O(1)	O(n)
Display	O(n + M)	O(n + M)
Load Factor	O(n + M)	O(n + M)
Rehashing	O(n)	O(n)

SPACE COMPLEXITY: O(n + M)

Error Table:

S.No	Errors Obtained	Rectification
01	Attribute Error	Check the linked list node attributes properly.
02	Index Error	Check the hash table index before accessing it.